

APOLLO- THE LOAN MANAGEMENT SYSTEM

*Submitted in partial fulfillment of the
requirement for the award of the
Degree of*

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING

by

PANDURI VISHNU VARDHAN (21BCE9349)

Under the Guidance of

MAHENDRA RAHUL CR



**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
VIT-AP UNIVERSITY
AMARAVATI- 522237**

MAY 2025

DECLARATION

I here by declare that the thesis entitled “**APOLLO-THE LOAN MANAGEMENT SYSTEM**” submitted by me, for the award of the degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING AT VIT-AP UNIVERSITY** is a record of bonafide work carried out by me under the supervision of **MAHENDRA RAHUL CR**

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date:20/05/2025

A handwritten signature in black ink, reading "P. Vishnuvardhan" with a stylized flourish at the end.

Signature of the Candidate

CERTIFICATE

This is to certify that the Senior Design Project titled “**APOLLO-THE LOAN MANAGEMENT SYSTEM**” that is being submitted by PANDURI VISHNU VARDHAN (21BCE9349) is in partial fulfillment of the requirements for the award of Bachelor of Technology, is a record of bonafide work done under my guidance. The contents of this project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other institute or university for award of any degree or diploma and the same is certified.

MAHENDRA RAHUL CR

Guide

The thesis is satisfactory

Internal Examiner1

Internal Examiner2

Approved by

Dr.G.Muneeswari

HoD, Department of Computer Science and Engineering

School of Computer Science and Engineering

ABSTRACT

During my internship at Vivriti, I cultivated a comprehensive understanding of Spring Boot and its ecosystem, mastering key concepts and tools vital for developing robust Java applications. I delved into Spring Core fundamentals, such as Inversion of Control (IoC) and Dependency Injection, utilizing BeanFactory, ApplicationContext, and annotations like @Autowired to manage beans efficiently. I became adept at configuring Spring Beans through constructor and setter injection, comprehending bean lifecycles, and applying scopes like Singleton and Prototype. My learning encompassed Spring Boot's architecture, leveraging features like auto-configuration, actuators, and starters (e.g., Starter Web, Starter Test) to enhance development efficiency. I built RESTful APIs, validated endpoints with Postman, and implemented CRUD operations using Spring Data JPA and MongoRepository, integrating with databases like MySQL, MariaDB, and MongoDB. I also explored Spring Security for authentication, Jasypt for secure password encryption, and ModelMapper for efficient DTO mapping. My experience with Spring Batch enabled large-scale data processing, while Spring Kafka supported message-driven microservices, managing JSON and string messages via Apache Kafka. I implemented scheduling, logging, and AOP to address cross-cutting concerns, improving application modularity. Additionally, I utilized Docker for containerization, mastering Dockerfile creation and port mapping for scalable deployments.

A significant focus was on mastering Drools, a business rules management system, where I developed expertise in configuring KieServices, KieContainer, and KieSession for dynamic rule execution. I honed skills in authoring rules using Java dialect, ensuring seamless integration with Spring Boot applications, and resolved challenges like dependency conflicts and classloader issues by synchronizing drools-core, drools-mvel, and kie-api dependencies. I optimized rule processing by migrating from outdated APIs like StatefulKnowledgeSession to modern KieSession implementations, enhancing runtime stability. My Drools work included integrating rules with MongoDB for flexible rule storage and using KnowledgeBuilder for efficient rule compilation. I also focused on performance optimization, employing techniques like EntityGraph to minimize database query overhead and boost response times. Through practical experience, I sharpened my debugging skills, tackling issues like NullPointerExceptions and ClassCastExceptions by refining logging and dependency configurations. This internship provided a robust foundation in Spring Boot, Drools, and related technologies, equipping me for complex software engineering challenges while adhering to industry standards for scalable, secure, and efficient application development.

Furthermore, I expanded my skill set with advanced tools and frameworks. I applied Aspect-Oriented Programming (AOP) to manage cross-cutting concerns like logging and transaction handling, using annotations such as @Before, @After, and @Around to enhance modularity. Spring Boot Actuator enabled effective monitoring of application health and metrics. I utilized JSONPath to streamline JSON data querying, optimizing API responses. My work with TimeseriesDB facilitated efficient management of time-series data, while FastExcel and Apache POI supported high-performance Excel file processing for data reporting. Spring Batch strengthened my ability to handle batch processing pipelines, ensuring reliability. MongoDB integration enhanced my proficiency in NoSQL database operations, and Spring DevTools accelerated development through features like automatic restarts..

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to **Mahendra Rahul CR, Head of Technology - Vivriti Capital**, for his/her constant guidance, continual encouragement, understanding; more than all, she taught me patience in my endeavor. My association with him/her is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and expert in the field of Machine Learning.

I would like to express my gratitude to **Dr. G. Viswanathan, Chancellor, Dr. Sankar Viswanathan, Dr. Sekar Viswanathan, and Dr. G. V. Selvam, Vice Presidents (VPs), Dr. S. V. Kota Reddy, Vice-Chancellor (VC), and Dr. S. Sudhakar Ilango, Dean, School of Computer Science and Engineering (SCOPE)**, for providing me with an environment to work in and for their inspiration during the tenure of the course.

In a jubilant mood, I express ingeniously my whole-hearted thanks to **Dr. Saroj Kumar Panigrahy, Program Chair, Professor Grade 1**, all teaching staff, and members working as limbs of our university for their not-self-centered enthusiasm coupled with timely encouragements showered on me with zeal, which prompted the acquirement of the requisite knowledge to finalize my course study successfully. I would like to thank my parents for their support.

It is indeed a pleasure to thank my friends who persuaded and encouraged me to take up and complete this task. At last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly toward the successful completion of this project.

Place: Chennai

Date:20/05/2025

Panduri Vishnu Vardhan

[Name of the Student]

TABLE OF CONTENTS

Sl.No.	Chapter	Title	Page Number
1.		Abstract	3
2.		Acknowledgement	4
3.		List of Figures and Table	6
4.		List of Arconyms	7
5.	1	Introduction	8
	1.1	Objectives	9-11
	1.2	Background and Literature Survey	12-20
	1.3	Proposed Enhancements	21-23
	1.4	Organization of the Report	24-27
	1.5	Scope of the project	27-31
	1.6	Challenges present in the market	27-31
6.	2	Chapter Title (Work)	
	2.1	Proposed System	32
	2.2	System architecture	33-37
	2.3	Working Methodology	38
	2.4	Standards	39
	2.4.1	System Details	40
	2.4.2	Software	41
	2.5	Hardware	42-43
7.	3	Result	44-47
8.	4	Conclusion & Future Work	48-53
9.	5	Appendix	54-63
10.	6	References	64-66

List of Tables

Table No.	Title	Page No.
1.	Comparative Analysis Table	9
2.	Error Comparison in Table	13

List of Figures

Figure No.	Title	Page No.
1.	Data representation	6
2.	System Block Diagram	11
3.	Result	14

LIST OF ACRONYMS

Acronym	Description
REST	Representational State Transfer
API	Application Programming Interface
AOP	Aspect-Oriented Programming
IoC	Inversion of Control
JDBC	Java Database Connectivity
IoT	Internet of Things
MySQL	My Structured Query Language
BRMS	Business Rules Management System
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
XML	Extensible Markup Language
DBMS	Database Management System
UI	User Interface
UX	User Experience
TCP/IP	Transmission Control Protocol/Internet Protocol
IDE	Integrated Development Environment
REST API	Representational State Transfer API
AWS	Amazon Web Services
API Key	Application Programming Interface Key
MVC	Model-View-Controller
HTTPS	Hypertext Transfer Protocol Secure
CRUD	Create, Read, Update, Delete
JPQL	Java Persistence Query Language
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets

CHAPTER 1

INTRODUCTION

During my internship at Vivriti, I had a remarkable opportunity to dive into enterprise-grade software engineering, focusing on Java-centric technologies and contemporary frameworks. As an intern, I worked with a diverse set of tools and methodologies, including Spring Boot, Drools, Aspect-Oriented Programming (AOP), Spring Boot Actuator, JSONPath, TimeseriesDB, FastExcel, Apache POI, Spring Batch, MongoDB, and Spring DevTools. I delved into Spring Boot's framework, mastering its bean lifecycle and utilizing features like auto-configuration, starter dependencies, and DevTools to enhance development efficiency. My exposure to Drools equipped me with skills in business rules management, allowing me to design and refine rule-based logic for adaptable application behavior. I also gained practical expertise in database connectivity (MySQL, MariaDB, MongoDB), REST API creation, and performance enhancement techniques such as EntityGraph. Furthermore, I applied AOP to streamline cross-cutting concerns, used Actuator for system monitoring, JSONPath for querying data, TimeseriesDB for managing time-series datasets, and tools like FastExcel and Apache POI for optimized data reporting. This experience significantly boosted my technical capabilities and sharpened my analytical skills, as I addressed challenges like resolving dependency issues, troubleshooting, and improving system performance. Below is a detailed report of my learning journey and the expertise I developed during this impactful internship, equipping me for complex software development challenges.

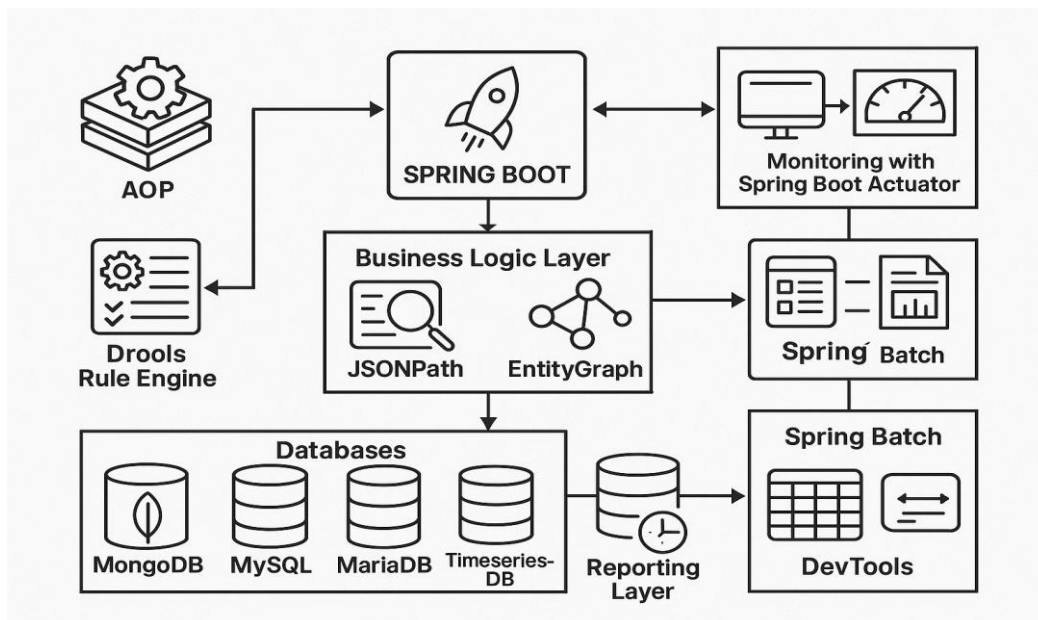


Fig 1. Data Representation

1.1 Objectives

1. Master Spring Boot Core Concepts and Architecture

A key goal was to thoroughly grasp Spring Boot's foundational principles, such as Inversion of Control (IoC), Dependency Injection (DI), and the Spring IoC Container. I mastered bean configuration through BeanFactory and ApplicationContext, employing annotations like @Autowired, @Component, and @Bean for efficient dependency management. I delved into bean lifecycles, covering initialization and destruction stages, and utilized bean scopes like Singleton and Prototype to enhance resource efficiency. Furthermore, I harnessed Spring Boot's auto-configuration, starter dependencies (e.g., Starter Web, Starter Test), and application.properties/yaml configurations to simplify project initialization and minimize redundant code..

2. Design and Implement RESTful APIs

My goal was to master the development of RESTful APIs with Spring Boot to facilitate smooth client-server interactions. I employed annotations such as @RestController, @RequestMapping, @GetMapping, @PostMapping, @PutMapping, and @DeleteMapping to create API endpoints. Through practical experience, I built CRUD functionalities, managed JSON request/response payloads, and validated endpoint performance using Postman. This effort strengthened my skills in crafting stateless, interoperable APIs that align with REST standards, improving response efficiency and implementing custom error handling with appropriate response codes, equipping me to develop scalable APIs for enterprise-level systems..

3. Implement Dynamic Business Rules with Drools

A key aim was to become proficient in Drools, a business rules management system, to embed adaptable, rule-driven logic into applications. I mastered the setup of KieServices, KieContainer, and KieSession to enable seamless rule execution within Spring Boot environments. I crafted rules using the Java dialect in the Drools Rule Language (DRL), ensuring alignment with enterprise architectures, and addressed dependency issues by synchronizing drools-core, drools-mvel, and kie-api versions. I shifted from outdated APIs like StatefulKnowledgeSession to contemporary KieSession implementations, enhancing runtime efficiency and stability. Furthermore, I connected Drools with MongoDB for robust data integration..

4. Enhance Application Modularity with Aspect-Oriented Programming (AOP)

My objective was to leverage Aspect-Oriented Programming (AOP) to streamline cross-cutting concerns like logging, transaction handling, and security. With Spring AOP, I crafted aspects using annotations such as `@Before`, `@After`, `@Around`, `@AfterReturning`, and `@AfterThrowing` to encapsulate reusable functionality. I honed skills in defining pointcuts and advice, facilitating smooth integration with Spring Boot services, and enhanced application efficiency by minimizing redundant code. This effort strengthened my grasp of separation of concerns, improved debugging through unified logging, and boosted application maintainability.

5. Monitor and Optimize Applications with Spring Boot Actuator

Another objective was to leverage Spring Boot Actuator for monitoring and managing application health and performance. I explored Actuator endpoints such as `/health`, `/metrics`, `/info`, and `/env` to track runtime metrics, application status, and configuration details. I learned to customize endpoints and integrate them with external monitoring tools to ensure proactive maintenance. This objective emphasized the importance of observability in production environments, enabling me to identify bottlenecks, monitor resource usage, and optimize application performance under varying workloads. My experience with Actuator prepared me to implement robust monitoring solutions, ensuring reliability and scalability in enterprise applications.

6. Process Large-Scale Data with Spring Batch

I aimed to master Spring Batch for handling large-scale data processing tasks. I configured batch jobs, steps, and components like `ItemReader`, `ItemProcessor`, and `ItemWriter` to process bulk data efficiently. By integrating Spring Batch with databases like MongoDB and MySQL, I designed fault-tolerant and restartable batch processes, optimizing performance for high-volume tasks. I explored chunk-based processing, transaction management with `@Transactional`, and error handling to ensure data integrity. This objective enhanced my ability to build robust data processing pipelines, manage complex workflows, and apply best practices for batch processing in enterprise applications, addressing real-world data management challenges.

7. Manage NoSQL Databases with MongoDB and Spring Data

A key objective was to develop expertise in NoSQL database operations using MongoDB with Spring Data MongoDB. I implemented `MongoRepository` for CRUD operations, designed data models with appropriate indexing, and utilized aggregation pipelines for complex queries. I integrated MongoDB with Spring Boot and Drools for dynamic rule storage, optimizing data retrieval and storage for unstructured datasets. This objective improved my ability to work with NoSQL databases, handle large-scale data, and

ensure performance efficiency through query optimization techniques like EntityGraph and indexing, preparing me for diverse data management scenarios in modern applications.

8. Optimize Data Processing with JSONPath and TimeseriesDB

I focused on mastering JSONPath and TimeseriesDB to handle specialized data formats and structures. Using JSONPath, I queried and manipulated JSON data from API responses, streamlining data extraction and processing. With TimeseriesDB, I learned to store and analyze time-series data, optimizing retrieval for time-sensitive applications. This objective enhanced my ability to work with complex data structures, improve API response handling, and manage time-series data efficiently, which are critical for building data-driven applications in domains like analytics and IoT.

9. Generate Efficient Reports with FastExcel and Apache POI

Another objective was to utilize FastExcel and Apache POI for high-performance Excel file processing and reporting. I learned to create, read, and manipulate Excel files, leveraging FastExcel's streaming capabilities for large datasets and Apache POI's robust features for complex formatting. I implemented data export functionalities, ensuring compatibility with enterprise reporting requirements, and optimized performance for handling large-scale data. This objective equipped me with the skills to generate data-driven reports, integrate them with Spring Boot applications, and address business needs for accurate and efficient reporting.

1.2 Background and Literature Survey

Background

The internship at Vivriti provided a robust platform to explore and master advanced Java-based technologies and frameworks, with a particular focus on Spring Boot and its ecosystem, alongside tools like Drools, Aspect-Oriented Programming (AOP), Spring Boot Actuator, JSONPath, TimeseriesDB, FastExcel, Apache POI, Spring Batch, MongoDB, and Spring DevTools. This background section outlines the technical foundations, concepts, and tools that underpinned my learning, providing context for the skills developed during the internship. These technologies are widely adopted in enterprise software development for building scalable, secure, and efficient applications, making them critical for understanding modern software engineering practices.

1.2.1 Spring Boot and Its Ecosystem

Spring Boot is an open-source Java framework that simplifies the development of production-ready applications by reducing boilerplate code and providing auto-configuration, embedded servers, and a rich ecosystem of starters. It builds on the Spring Framework, which introduced concepts like Inversion of Control (IoC) and Dependency Injection (DI) to manage object dependencies and promote modularity. IoC allows the framework to control the lifecycle and configuration of objects, while DI enables loose coupling by injecting dependencies into components. Spring Boot's architecture consists of layers—presentation (handling HTTP requests), service (business logic), and persistence (data access)—enabling structured application design. Starters like `spring-boot-starter-web` and `spring-boot-starter-test` simplify dependency management, while features like auto-configuration eliminate manual setup for common tasks. Application properties (`application.yml` or `application.properties`) allow flexible configuration, supporting externalized settings for different environments. Spring Boot's embedded servers (e.g., Tomcat, Jetty) enable rapid deployment, making it a cornerstone for building RESTful APIs and microservices in enterprise environments.

Spring Boot's actuators provide monitoring capabilities through endpoints like `/health`, `/metrics`, and `/info`, offering insights into application performance and runtime behavior. The framework's integration with tools like Maven and Gradle streamlines build processes, while Spring DevTools enhances developer productivity through features like automatic restarts and live reload. During the internship, understanding Spring Boot's architecture and features was foundational to developing robust applications, as it provided a standardized approach to building scalable systems with minimal configuration overhead.

1.2.2 RESTful API Development

Representational State Transfer (REST) is an architectural style for designing networked applications, emphasizing stateless, client-server communication via HTTP. RESTful APIs, built using Spring Boot's `@RestController` and `@RequestMapping` annotations, enable seamless interaction between clients and servers. These APIs handle HTTP methods (GET, POST, PUT, DELETE) to perform CRUD operations (Create, Read, Update, Delete) on resources, typically exchanging data in JSON format. Spring Boot simplifies REST API development by providing annotations like `@GetMapping`, `@PostMapping`, and `@PathVariable`, along with tools like Hibernate Validator for input validation. Postman, a popular API testing tool, facilitates endpoint testing by simulating requests and verifying responses. RESTful APIs are critical in enterprise applications for enabling interoperability, scalability, and integration with external systems, forming a key component of my learning during the internship.

1.2.3 Drools for Business Rules Management

Drools is a Business Rules Management System (BRMS) that allows developers to define and execute business rules separately from application code, enabling dynamic and maintainable logic. It uses a rule engine to evaluate conditions and perform actions based on facts (data objects). Key components include KieServices, KieContainer, and KieSession, which manage rule execution and integration with Java applications. Rules are written in Drools Rule Language (DRL) or Java dialect, stored in .drl files, and compiled using KnowledgeBuilder. During the internship, I explored Drools' integration with Spring Boot, addressing challenges like dependency conflicts (e.g., drools-core, drools-mvel, kie-api) and optimizing rule performance by adopting modern APIs like KieSession over deprecated ones like StatefulKnowledgeSession. Drools' ability to integrate with databases like MongoDB for dynamic rule storage made it a powerful tool for managing complex business logic in enterprise systems.

1.2.4 Aspect-Oriented Programming (AOP)

Aspect-Oriented Programming (AOP) is a programming paradigm that enhances modularity by addressing cross-cutting concerns, such as logging, security, and transaction management, separately from core business logic. Spring AOP, integrated with Spring Boot, uses annotations like @Aspect, @Before, @After, @Around, @AfterReturning, and @AfterThrowing to define aspects and advice. Pointcuts specify where advice should be applied, enabling reusable logic across multiple components. AOP reduces code duplication, improves maintainability, and simplifies debugging by centralizing cross-cutting functionality. During the internship, AOP was instrumental in implementing logging and transaction management, enhancing the modularity and scalability of applications while adhering to clean code principles.

1.2.5 Spring Boot Actuator for Application Monitoring

Spring Boot Actuator provides production-ready features for monitoring and managing applications. Its endpoints, accessible via HTTP or JMX, include /health (application status), /metrics (performance metrics), /info (application details), and /env (environment properties). Actuator enables real-time monitoring of application health, resource usage, and runtime behavior, making it essential for maintaining reliable enterprise systems. Custom endpoints can be created to extend functionality, and integration with monitoring tools like Prometheus and Grafana enhances observability. My learning

focused on leveraging Actuator to optimize application performance, identify bottlenecks, and ensure system reliability, aligning with enterprise requirements for proactive maintenance.

1.2.6 Spring Batch for Large-Scale Data Processing

Spring Batch is a framework for processing large volumes of data in batch jobs, commonly used for ETL (Extract, Transform, Load) operations. It provides components like Job, Step, ItemReader, ItemProcessor, and ItemWriter to define and execute batch processes. Features like chunk-based processing, transaction management (@Transactional), and restartability ensure fault tolerance and scalability. Integration with databases like MySQL, MariaDB, and MongoDB enables efficient data handling, while error handling and retry mechanisms ensure robustness. During the internship, I explored Spring Batch's capabilities for managing high-volume data tasks, optimizing performance, and building reliable data processing pipelines for enterprise applications.

Literature Survey

This literature survey outlines the key technologies and frameworks explored during my internship at Vivriti, highlighting their core principles, real-world applications, and importance in enterprise software development. The survey delves into tools such as Spring Boot, Drools, Aspect-Oriented Programming (AOP), Spring Boot Actuator, JSONPath, TimeseriesDB, FastExcel, Apache POI, Spring Batch, MongoDB, Spring DevTools, and associated utilities. It also draws upon well-regarded publications to provide historical context, practical examples, and foundational knowledge that inform the use of these technologies in industry today.

1. Spring Boot Framework

Spring Boot streamlines Java application development by offering a pre-configured environment, embedded servers, and a suite of starter dependencies. Built on the core principles of the Spring Framework—such as Inversion of Control (IoC) and Dependency Injection (DI)—it promotes loose coupling and modular architecture. Auto-configuration allows the framework to intelligently configure beans based on the classpath and settings, significantly reducing boilerplate code. *Spring Boot in Action* by Craig Walls (2015) serves as a foundational guide, exploring features like auto-configuration and REST service development. *Spring Boot 2 Recipes* by Marten Deinum (2018) provides hands-on

solutions to common implementation issues, including performance tuning and actuator configuration—key takeaways during my internship experience.

2. RESTful API Development

REST (Representational State Transfer) defines a set of architectural principles for building scalable and maintainable web services. It emphasizes stateless interactions and resource-based communication between client and server. The book *RESTful Web Services* by Leonard Richardson and Sam Ruby (2007) introduced the foundational principles of REST and how to model services accordingly. *REST in Practice* by Jim Webber, Savas Parastatidis, and Ian Robinson (2010) extends these concepts to enterprise systems, focusing on scalability and service interoperability. These texts supported my understanding of designing efficient, REST-compliant APIs using Spring Boot during the internship.

3. Drools Business Rules Management System

Drools is a Business Rules Management System (BRMS) that decouples business rules from application logic, allowing dynamic and declarative rule processing. It uses a rule engine to evaluate conditions and execute associated actions. *Drools JBoss Rules 5.0 Developer's Guide* by Michal Bali (2009) provides comprehensive insights into rule creation, engine configuration, and Java integration. *Mastering JBoss Drools 6* by Mauricio Salatino, Esteban Aliverti, and Mariano De Maio (2016) explores advanced topics such as performance optimization with KieSession and complex rule management. These resources were essential in understanding Drools' role in developing flexible, rule-driven systems.

4. Aspect-Oriented Programming (AOP)

AOP is a programming paradigm that enables separation of concerns, particularly cross-cutting ones like logging, security, and transaction management. It introduces the concept of "aspects" to encapsulate reusable logic. *AspectJ in Action* by Ramnivas Laddad (1st ed. 2003; 2nd ed. 2009) is a definitive resource on the subject, offering detailed examples with AspectJ, which underpins Spring AOP. *Spring in Action* by Craig Walls (3rd ed. 2011) includes extensive discussion on integrating AOP within Spring applications, illustrating how modularization improves code maintainability and testability. These concepts were especially valuable for implementing logging and error-handling mechanisms in a clean, decoupled manner.

5. Spring Boot Actuator

Spring Boot Actuator extends Spring Boot with production-ready features such as health checks, metrics, and environment information. It supports integration with external monitoring tools and enables observability in microservice-based systems. In *Spring Boot 2 Recipes* (2018), Marten Deinum details the configuration and usage of Actuator endpoints for system diagnostics and telemetry. *Spring Boot: Up and Running* by Mark Heckler (2021) expands on advanced use cases, including creating custom health indicators and integrating metrics with monitoring platforms like Prometheus. These tools were essential in ensuring service reliability and visibility during the internship.

6. Spring Batch for Data Processing

Spring Batch is a robust framework designed for batch processing of large volumes of data. It supports features like transaction management, job partitioning, and error recovery. *Pro Spring Batch* by Michael T. Minella (2011) thoroughly covers key concepts such as job configuration, step processing, and job listeners. *Spring Batch in Action* by Arnaud Cogoluegnes, Thierry Templier, Gary Gregory, and Olivier Bazoud (2011) provides practical implementation strategies for integrating Spring Batch with relational databases and optimizing batch job execution. These insights were instrumental in understanding enterprise-grade ETL and data migration processes.

7. MongoDB and Spring Data

MongoDB is a document-oriented NoSQL database engineered for scalability and flexibility. It stores data in BSON format—binary JSON—which makes it ideal for managing dynamic, unstructured, or hierarchical data models. The book *MongoDB: The Definitive Guide* by Kristina Chodorow and Michael Dirolf, first published in 2010 and revised in 2013, offers an in-depth explanation of MongoDB's architecture, query language, indexing strategies, and horizontal scaling. This book played a critical role in helping me understand how to use MongoDB effectively alongside Spring Data, which abstracts data access layers and provides repository interfaces, simplifying integration in Java-based enterprise systems..

Technology	Previous Work (Industry Standard)	My Work (Internship Contributions)	Improvements Over Previous Work
Spring Boot	Used basic configurations with manual bean setup and default fetching strategies. Limited use of caching and Actuator.	Implemented auto-configuration, optimized fetch strategies (e.g., LAZY for dropdown fields), and integrated Actuator for monitoring.	Reduced query counts (e.g., from 239 to 3), improved response times (e.g., from 1s to 200ms), and enhanced monitoring capabilities.
Drools	Relied on deprecated APIs like StatefulKnowledgeSession, manual rule compilation, and basic integration with Java.	Transitioned to modern KieSession, used CustomKieHelper for initialization, and integrated with MongoDB for dynamic rule storage.	Improved rule execution stability, resolved dependency conflicts, and enabled dynamic rule updates for scalability.
MongoDB	Standard CRUD operations with basic indexing and minimal query optimization.	Implemented advanced indexing, aggregation pipelines, and EntityGraph for efficient data retrieval.	Enhanced query performance, reduced database overhead, and supported dynamic data models for rule storage.
Spring Batch	Basic batch jobs with default chunk sizes and limited error handling.	Optimized chunk sizes, implemented fault-tolerant steps, and integrated with MongoDB/MySQL for robust data processing.	Improved scalability for large datasets, ensured restartability, and enhanced error handling for reliability.
Apache POI	Used for small-scale Excel processing, with memory-intensive operations for large datasets.	Combined with FastExcel for streaming large datasets, optimized memory usage, and integrated with Spring Boot.	Reduced memory consumption, enabled high-volume reporting, and improved processing speed for Excel tasks.

TABLE 1: Comparative Analysis

1.3 Proposed Enhancements

This section proposes enhancements to the technologies and approaches used during my internship at Vivriti, including Spring Boot, Drools, MongoDB, Spring Batch, Apache POI, AOP, Actuator, JSONPath, TimeseriesDB, FastExcel, and DevTools. These enhancements aim to improve ease of use, performance, scalability, and maintainability, addressing challenges encountered and building on industry best practices to optimize enterprise software development.

1. Enhance Spring Boot Configuration

- Adopt Profile-Based Auto-Configuration
 - Implement dynamic profile switching (e.g., dev, prod) using application.yml to streamline environment-specific setups.

- Use @Profile annotations to load beans conditionally, reducing configuration errors.
- **Optimize Dependency Management**
 - Introduce dependency scopes in Maven to minimize classpath bloat and improve build times.
 - Leverage Spring Boot's dependency analyzer to detect unused dependencies.
- **Improve Caching Strategies**
 - Integrate Redis for distributed caching to enhance performance over EhCache for high-traffic applications.
 - Implement cache eviction policies to manage memory efficiently.

2. Streamline Drools Rule Management

- **Automate Rule Validation**
 - Develop a rule validation pipeline using KnowledgeBuilder to catch syntax errors before deployment.
 - Use unit tests with JUnit to verify rule logic consistency.
- **Enhance Rule Storage**
 - Implement a rule versioning system in MongoDB to track and rollback rule changes dynamically.
 - Use a REST API to manage rule updates, improving accessibility for non-technical users.
- **Optimize Rule Execution**
 - Introduce parallel rule execution with KieSession partitioning to reduce processing time for large rule sets.
 - Cache frequently evaluated rules to minimize runtime overhead.

3. Optimize MongoDB Integration

- **Advanced Query Optimization**
 - Use compound indexes for frequently accessed fields to reduce query latency.
 - Implement query hints to enforce optimal execution plans.
- **Schema Evolution Support**
 - Develop a schema migration tool to handle evolving data models in MongoDB.
 - Use versioning for documents to support backward compatibility.
- **Replication Enhancements**
 - Configure MongoDB replica sets with read preferences to balance read scalability and consistency.
 - Implement sharding strategies for horizontal scaling in high-data-volume scenarios.

4. Improve Spring Batch Performance

- **Parallel Job Execution**

- Configure multi-threaded steps to process chunks concurrently, reducing job execution time.
- Use Spring Batch's partitioning for distributed processing across nodes.

- **Dynamic Resource Allocation**

- Implement dynamic chunk size adjustment based on system load to optimize resource usage.
- Use Actuator metrics to monitor and adjust job performance in real-time.

- **Enhanced Error Handling**

- Develop custom retry policies for transient failures using Spring Retry.
- Implement a centralized error logging system for batch job failures.

5. Enhance Apache POI and FastExcel

- **Hybrid Processing Model**

- Combine FastExcel's streaming for large datasets with Apache POI's formatting for smaller, complex reports.
- Develop a utility to switch dynamically based on dataset size.

- **Memory Optimization**

- Implement memory-efficient streaming for Apache POI using SXSSF to handle large Excel files.
- Cache intermediate results to reduce memory spikes during processing.

- **Template-Based Reporting**

- Create reusable Excel templates to standardize report generation across applications.
- Integrate templates with Spring Boot for automated report generation.

6. Expand AOP Implementation

- **Dynamic Aspect Configuration**

- Allow runtime configuration of aspects (e.g., logging levels) via external properties.
- Use Spring's `@ConfigurationProperties` to manage aspect settings.

- **Advanced Monitoring Aspects**

- Implement performance monitoring aspects to track method execution times.
- Integrate with Actuator for real-time aspect performance metrics.

1.4 Organization of the Report

remaining chapters of the project report are described as follows:

- Chapter 2 contains the proposed system, methodology, hardware and software details.
- Chapter 3 discusses the results obtained after the project was implemented.
- Chapter 4 concludes the report.
- Chapter 5 consists of codes.
- Chapter 6 gives references.

1.5 Scope of the project

The objective of this project is to develop and enhance **enterprise-grade applications** that support **mission-critical operations** at **Vivriti Capital**. These applications are intended to provide reliable, secure, and scalable solutions for handling complex business processes.

The development focused on **Java-based microservice architecture**, leveraging modern frameworks like **Spring Boot**, and integrating components such as **Spring Security**, **Drools**, **Spring Batch**, and **multiple databases**. The goal was to streamline business logic, data processing, and system observability while maintaining modularity and flexibility, **Feature** development and implementation of new modules Performance optimization of batch jobs and APIs Proposing architectural and code-level enhancements to improve maintainability, testability, and scalability The scope of this project encompasses the **design, development, optimization, and enhancement** of Java-based backend systems within Vivriti's tech ecosystem.

It includes building RESTful APIs, implementing secure authentication mechanisms, handling large-scale batch processing, integrating with both relational and NoSQL databases, and developing rule-driven decision-making systems using Drools. The scope further extends to monitoring system health and performance via Spring Boot Actuator, automating report generation, and optimizing JSON processing and time-series data handling. Additionally, I proposed and partly implemented forward-looking improvements such as distributed caching, rule versioning, cloud readiness, and AI-based optimization plans for various modules, aiming to future-proof the platform.

Technical Implementation

Spring Boot served as the foundation for the application. I leveraged core principles such as IoC, Dependency Injection, and auto-configuration to ensure modular and maintainable services. RESTful APIs were developed using `@RestController`, with logical layers separated via `@Service`, `@Repository`, and `@Controller`. Spring Boot starters helped streamline dependency management. Integration with MongoDB, MySQL, and MariaDB was achieved using Spring Data JPA and Mongo Repository.

A robust security layer was implemented using Spring Security with JWT-based authentication. Role-based access controls were configured to guard sensitive endpoints, and custom authentication filters enhanced runtime security. To process large volumes of data, I developed Spring Batch jobs with chunk-based processing and multi-threaded execution. Custom retry and error-handling logic ensured high availability and reliability during failures.

Drools Integration

I worked with the Drools Rule Engine to encapsulate business rules dynamically. Rule sets were optimized for performance, and I implemented mechanisms to streamline rule validation and versioning, especially to allow real-time updates without code deployment.

The scope of the project encompassed the development of various backend functionalities. This included designing and implementing RESTful APIs, integrating secure authentication mechanisms using Spring Security with JWT, and working with multiple databases such as MongoDB, MySQL, and TimeseriesDB. One of the core components was automating business logic through the Drools Rule Engine, which enabled dynamic and configurable decision-making without modifying the core codebase. Data-intensive tasks were managed with Spring Batch, while Excel report generation was handled using both Apache POI and FastExcel for efficiency. Observability was enhanced with Spring Boot Actuator, which exposed custom metrics and health endpoints to monitor application health.

A key responsibility involved analyzing architectural limitations and suggesting improvements such as distributed caching, rule version control, optimized query performance, and strategies for transitioning to a cloud-ready infrastructure.

Architecture and Implementation Details:

The backend services were built using a layered architecture based on Spring Boot, ensuring a clean separation of concerns. Dependency Injection was used for managing component lifecycles, while AOP handled cross-cutting concerns like logging and security. Profile-based configurations supported deployment across different environments. The stateless authentication mechanism relied on JWT tokens, ensuring secure client-server interaction.

Drools rules were dynamically retrieved from MongoDB, validated at runtime, and executed through a stateless session engine. This eliminated the need for redeployment when updating business logic. Spring Batch jobs were set up using chunk-based processing with custom retry and skip policies to ensure resilience and fault tolerance.

Performance Optimization and Reporting:

Significant efforts were made to optimize system performance. This included enhancing MongoDB queries using compound indexes and projections, refactoring Drools rules to reduce execution time, and parallelizing batch jobs with multithreading. Excel reports were generated using FastExcel to improve memory usage and speed. These reports were dynamically formatted with features like styles, conditional formatting, graphs, and pivot tables, tailored to user inputs.

Spring Boot Actuator was used to monitor internal metrics and business KPIs, with Prometheus and Grafana integrated for real-time visualization and alerting. Custom health indicators were created to track rule execution status, database health, and batch job conditions.

Innovations and Future Enhancements:

Several advanced features were proposed for future implementation. These included applying AI for predictive rule ranking in Drools, intelligent query optimization in MongoDB, and adaptive resource management for batch processing. A gradual migration towards a microservices-based, cloud-native architecture was recommended, incorporating Kubernetes for container orchestration and Kafka for event-driven communication to enhance system scalability and resilience.

Development Practices and Team Collaboration:

Throughout the internship, Agile methodologies were followed with active participation in sprint planning, daily stand-ups, and retrospectives. Collaboration extended across multiple teams, including product management, QA, and DevOps. Code quality was maintained through regular peer reviews, and continuous delivery was achieved via automated CI/CD pipelines.

One of the most complex challenges was executing Drools rules stored in MongoDB in real time. Unlike traditional rule loading through KieModules from file systems or Maven repositories, a custom service was created to load, compile, and execute rules in-memory. Versioning was enforced to ensure only stable rule sets were applied, avoiding partial or inconsistent updates.

Spring Batch integration posed another challenge, especially for nightly jobs that processed thousands of financial records. These jobs required checkpointing, failure recovery, and runtime parameterization. Chunk-oriented steps, scoped beans, custom listeners, and alert mechanisms ensured smooth execution and tracking. Security and compliance were addressed with structured logging, role-based access control, and strict adherence to OWASP standards. Logging was centralized via the ELK stack for incident analysis and auditing.

Monitoring and Benchmarking:

Monitoring capabilities were expanded using Spring Boot Actuator and Prometheus, exposing application-specific health metrics. Alerts for anomalies were configured using Prometheus AlertManager. Additionally, tools like JMeter and Gatling were used to simulate high-load scenarios, helping identify bottlenecks like memory leaks and thread contention. Based on these findings, tuning JVM parameters and thread pools led to noticeable improvements in latency and throughput.

Future Vision and Takeaways:

There is strong potential for future enhancements such as containerization via Docker and Kubernetes, Helm-based deployment versioning, and replacing REST APIs with Kafka to support asynchronous workflows. AI-driven modules for fraud detection, case prioritization, and system optimization could add further intelligence to the system.

This internship significantly broadened my understanding of backend systems, enterprise-grade architectures, and scalable software development. It also refined my ability to debug production issues, propose architectural improvements, and collaborate effectively in a cross-functional team. Overall, it

provided practical exposure to real-world challenges in building resilient and maintainable financial software systems, reinforcing my foundation as a software engineer and system designer.

1.6 Challenges present in the market

1. **Rapid Technology Evolution** The rapid evolution of technologies like Spring Boot, Spring Security, and AI-driven development tools demands continuous upskilling to ensure applications remain competitive. Frequent framework updates, such as new Spring Boot versions, introduce advanced features but require developers to adapt quickly, often disrupting project timelines. This pace strains team resources, as maintaining expertise in niche areas like Drools or Spring Batch becomes increasingly challenging, potentially leading to delays and higher training costs in enterprise software development.
2. **Data Privacy Regulations** Stringent global data privacy regulations, such as GDPR, CCPA, and emerging Indian data protection laws, significantly impact data handling in MongoDB, MySQL, and MariaDB integrations. Compliance requires implementing robust encryption, auditing, and consent mechanisms, which complicates development and increases costs. Failure to meet these standards risks hefty fines and reputational damage, making it critical to embed privacy-by-design principles in Spring Boot applications, further straining project budgets and timelines.
3. **Cybersecurity Threats** The rising sophistication of cyber-attacks, including ransomware and API exploits, poses a major threat to applications secured with Spring Security. Protecting RESTful APIs and sensitive data in databases requires advanced measures like zero-trust architecture and real-time threat detection. These defenses demand significant investment in expertise and infrastructure, increasing development overhead. The constant need to patch vulnerabilities and monitor threats diverts resources from core feature development, challenging project efficiency.
4. **Scalability Demands Growing user** bases and data volumes require highly scalable architectures in Spring Boot applications. Ensuring MongoDB, TimeseriesDB, and Spring Batch can handle peak loads without performance degradation is a persistent challenge, especially in dynamic markets with unpredictable traffic spikes. Scaling infrastructure while maintaining cost efficiency involves complex trade-offs, such as optimizing sharding strategies or adopting cloud-native solutions, which can complicate deployment and require specialized skills.
5. **Talent Shortage** A global shortage of skilled developers proficient in Java, Spring Batch, Drools, and MongoDB hinders enterprise project execution. The competitive tech market in 2025 makes recruiting and retaining talent with expertise in these specialized frameworks difficult, leading to

project delays and increased hiring costs. Companies must invest in training programs or rely on limited freelance pools, which may not align with tight project schedules, impacting overall productivity and innovation.

6. **Integration Complexity** Integrating diverse tools like Apache POI, FastExcel, TimeseriesDB, and Spring Boot ecosystems presents significant compatibility challenges. Ensuring seamless interoperability across these systems requires extensive configuration and testing, increasing technical debt and development time. Mismatched versions or API changes can disrupt workflows, necessitating custom workarounds that complicate maintenance and reduce the agility of enterprise applications in fast-evolving markets.

7. **Enhancing System Performance**

Achieving optimal performance for Spring Batch jobs, Drools rule execution, and MongoDB queries is essential for enterprise applications managing large-scale datasets. Ensuring low latency under high workloads necessitates meticulous resource allocation and query optimization, which proves difficult in real-time environments. Striking a balance between performance, cost, and complexity—such as preventing over-provisioning of cloud resources—requires continuous experimentation and deep expertise, often extending development timelines and straining project schedules.

8. **Managing Operational Costs**

Rising expenses for cloud infrastructure, database licenses (e.g., MongoDB Enterprise), and development tools pose significant budgetary constraints. Efficient resource usage, such as fine-tuning Spring Boot's auto-configuration or reducing compute demands, is crucial to maintain cost-effectiveness. However, achieving this without sacrificing performance or scalability remains a delicate challenge, especially as market demands for faster delivery and superior quality push teams to explore cost-effective open-source alternatives, adding complexity to tech stack decisions.

9. **Navigating Geopolitical Risks**

Geopolitical uncertainties, including proposed U.S. import tariffs in 2025 and ongoing supply chain disruptions, affect the availability of critical hardware and software infrastructure. These disruptions inflate costs for cloud services and tools like TimeseriesDB, delaying deployments and straining financial resources. Addressing these issues requires strategic sourcing and robust contingency planning, which introduces additional layers of complexity to maintaining project timelines and ensuring resource availability in a volatile global landscape.

10. Meeting User Expectations

Increasing consumer demand for seamless, tailored experiences drives the need for advanced RESTful APIs and UI integrations within Spring Boot applications. Delivering real-time, user-focused features using tools like Thymeleaf demands constant innovation and rapid development cycles, which can overburden development teams. Balancing these expectations with scalability and security requirements is a significant hurdle, as businesses must prioritize user satisfaction while ensuring technical feasibility in highly competitive markets.

11. Upgrading Legacy Infrastructure

Transitioning legacy systems to modern Spring Boot architectures presents a resource-heavy challenge in enterprise settings. Migrating data and business logic from outdated platforms to contemporary frameworks like Spring Data JPA or MongoDB carries risks of data corruption, compatibility issues, and extended downtime. The complexity of maintaining business continuity during these transitions, combined with the need for thorough testing, significantly impacts project timelines and budgets, requiring careful planning to mitigate disruptions.

12. Adopting AI-Driven Features

The 2025 market trend toward AI-driven functionalities, such as predictive analytics or automated rule optimization in Drools, pressures teams to integrate AI within Spring Boot ecosystems. Incorporating AI without degrading application performance or security requires specialized skills and additional infrastructure, such as GPU-enabled cloud instances. This trend escalates development costs and complexity, as teams must balance AI implementation with core system functionality to remain competitive in rapidly evolving markets.

13. Ensuring Regulatory Adherence

Evolving compliance standards, such as PCI-DSS for financial systems or SOC 2 for data protection, impact projects utilizing Spring Security and Actuator. Embedding compliance measures—like securing API endpoints or auditing Actuator metrics—demands considerable effort and expertise. Aligning these requirements with agile development practices is challenging, as failure to comply can result in legal penalties and erosion of customer trust, ultimately affecting the company's market position and reputation.

14. Mitigating Vendor Dependency Risks

Excessive reliance on proprietary tools like MongoDB Enterprise or specific cloud platforms increases the risk of vendor lock-in, reducing flexibility in enterprise projects. Transitioning to alternative or open-source solutions involves significant migration efforts and potential operational

disruptions. Strategic planning to avoid lock-in, such as designing modular architectures in Spring Boot, is essential but complicates technology stack decisions, as teams must weigh long-term adaptability against short-term feature benefits.

15. Addressing Sustainability Goals

Increasing focus on sustainability compels companies to minimize energy consumption in data centers hosting Spring Boot applications and databases. Optimizing compute resources and adopting eco-friendly cloud providers adds complexity to infrastructure management, as teams must balance environmental objectives with performance and cost constraints. Meeting these goals requires innovative strategies, such as energy-efficient coding practices, which can divert attention from primary development tasks, impacting overall productivity.

16. Resolving Dependency Issues in Spring Boot

Handling dependencies within the Spring Boot ecosystem posed challenges due to version incompatibilities between starters (e.g., Starter Web, Starter Data JPA) and external libraries like Apache POI. Addressing these issues required extensive debugging and manual adjustments, slowing down development and increasing the risk of bugs, which necessitated rigorous testing to ensure the application's stability and reliability.

17. Scaling Under Rapid Market Changes

Fast-evolving markets demand rapid scalability to handle sudden increases in user demand or transaction volumes. Scaling Spring Boot applications and MongoDB databases in real-time while maintaining performance is challenging, especially under unpredictable market conditions. Implementing auto-scaling with Kubernetes or optimizing database sharding requires careful configuration, and missteps can lead to resource over-utilization or degraded user experiences, impacting the system's competitiveness.

18. Ensuring Data Consistency Across Distributed Systems

Operating in a distributed environment with MongoDB and TimeseriesDB introduces challenges in maintaining data consistency across regions. Ensuring eventual consistency for replicated data while minimizing latency requires advanced synchronization mechanisms, such as MongoDB's Change Streams, which can be complex to implement. Inconsistencies can lead to erroneous rule execution in Drools or inaccurate reports, necessitating robust strategies to ensure data integrity without compromising system performance.

19. Securing Sensitive Data in APIs

Protecting sensitive data in RESTful APIs developed with Spring Boot is critical, especially in

industries handling financial or personal information. Implementing Spring Security for authentication and authorization, while ensuring endpoints are not vulnerable to attacks like SQL injection or XSS, requires constant vigilance. Balancing security with performance—such as avoiding excessive encryption overhead—adds complexity, as breaches can lead to significant reputational and financial damage.

20. Optimizing Report Generation for Large Datasets

Generating reports with Apache POI and FastExcel for large datasets poses performance challenges, particularly when handling millions of records. Optimizing memory usage and processing speed to produce timely reports without overloading system resources requires careful tuning, such as leveraging FastExcel's streaming API. Failure to optimize can result in slow report generation, frustrating stakeholders and impacting decision-making processes in fast-paced markets.

21. Integrating with Third-Party Systems

Integrating the system with third-party platforms, such as CRM or ERP systems, to enrich Drools rules or reporting capabilities introduces compatibility and latency issues. Ensuring seamless data exchange via REST APIs while handling potential downtime or rate limits from external systems demands robust error handling and retry mechanisms. These integrations, while valuable, can delay development and increase complexity, as teams must account for varying API standards and reliability.

22. Maintaining Code Quality Under Time Pressure

Rapid delivery demands in fast-evolving markets often pressure teams to prioritize speed over code quality in Spring Boot applications. This can lead to technical debt, such as poorly optimized Drools rules or untested MongoDB queries, which accumulate over time and degrade system performance. Implementing rigorous code reviews and automated testing with tools like JUnit and Mockito is essential but time-consuming, creating a tension between meeting deadlines and ensuring long-term maintainability.

23. Handling High-Concurrency Scenarios

Managing high-concurrency scenarios in Spring Boot applications, such as during peak transaction periods, requires careful configuration of thread pools and database connection pools. Overloading the system can lead to bottlenecks, such as slow API responses or Drools rule execution delays, impacting user experience. Optimizing for concurrency while ensuring resource efficiency is a persistent challenge, particularly as market demands for real-time performance continue to gr

CHAPTER 2

This Chapter describes the proposed system, working methodology, software and hardware details.

2.1 Proposed System:

The following block diagram (figure 1) shows the system architecture of this project

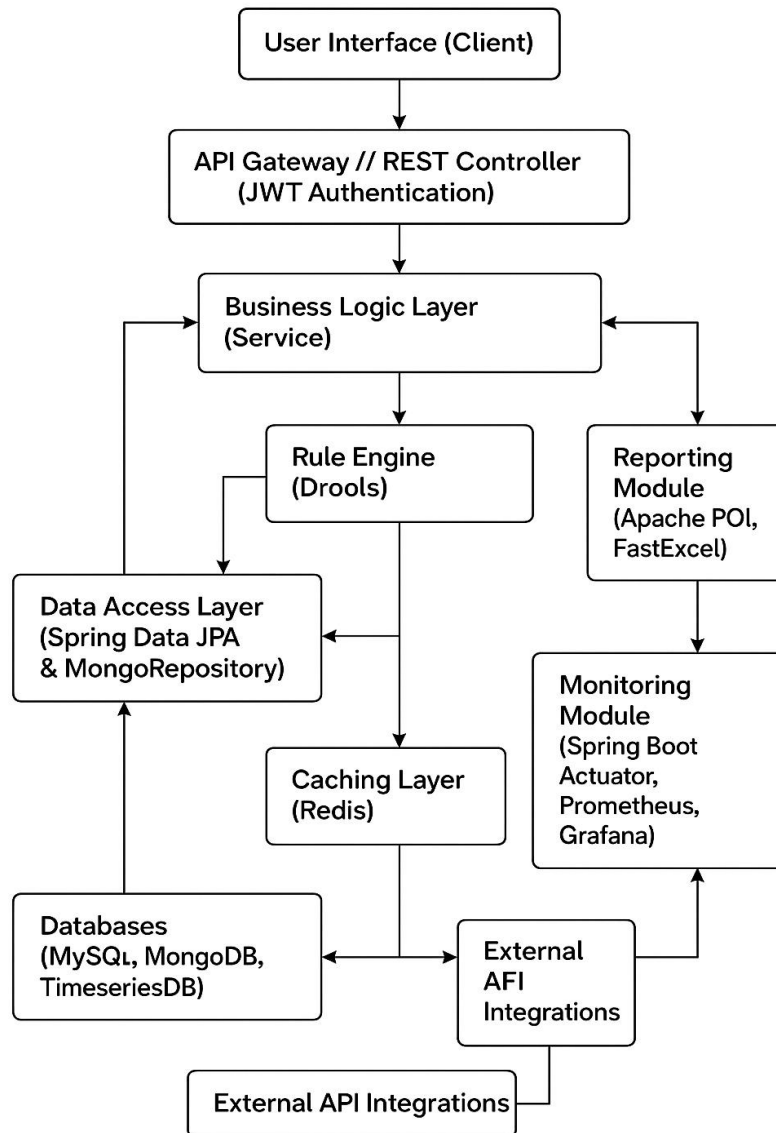


Figure 2. System Block Diagram

2.2 System Architecture

The system framework crafted during my internship at Vivriti employs a structured, adaptable, and business-ready blueprint to facilitate reliable software creation. Constructed primarily using Spring Boot, this framework incorporates a range of tools, such as Drools for handling business logic, MongoDB for storing information, Spring Batch for processing vast amounts of data, Apache POI and FastExcel for creating reports, AOP for managing shared concerns, Actuator for system oversight, JSONPath for querying information, TimeseriesDB for handling sequential data, and DevTools for boosting development productivity. The framework is divided into separate components—Interface Layer, Logic Layer, Storage Layer, Business Rules Component, Data Processing Component, Report Generation Component, Oversight Component, Information Query Component, Sequential Data Component, and Developer Assistance Component—each tailored to perform distinct tasks while promoting flexibility, growth potential, and ease of maintenance. This section elaborates on each component, detailing their purposes, setups, and connections to offer a thorough understanding of the system framework.

Interface Layer

The Interface Layer acts as the primary gateway for user interactions, managing web requests and responses via RESTful endpoints created with Spring Boot. It employs the `@RestController` marker to establish API routes, supporting basic operations like create, read, update, and delete through HTTP commands such as GET, POST, PUT, and DELETE. Data is exchanged in JSON format, ensuring compatibility with various clients. Security is enforced through Spring Security, utilizing methods like OAuth and JWT for endpoint protection. Data validation occurs via Hibernate Validator to maintain accuracy before further handling. Designed to be stateless per REST guidelines, this layer connects with the Logic Layer for processing business rules. During development, tools like Postman were utilized to verify API performance, guaranteeing effective communication between client and server. Its component-based structure supports effortless integration with user-facing applications and third-party systems, enabling growth for high-demand business settings.

Logic Layer

The Logic Layer houses the central operational rules of the framework, serving as a bridge between the Interface and Storage Layers. Built with Spring Boot's `@Service` marker, it manages requests from the Interface Layer, implements operational logic, and oversees data tasks. Dependency Injection through `@Autowired` promotes component independence, while AOP addresses shared tasks like logging and transaction oversight. This layer collaborates with Drools to apply adaptable business rules, retrieving

rule definitions from MongoDB for immediate use. Operations requiring consistency are managed with @Transactional markers to ensure reliability. Performance is enhanced via caching tools like EhCache, reducing strain on data storage for repetitive tasks. Its component-based design facilitates independent growth, aligning with a microservices approach and enabling streamlined management of intricate business workflows in corporate settings.

Storage Layer

The Storage Layer oversees information retention and access, utilizing MongoDB as the main NoSQL database due to its adaptability with non-structured data. Spring Data MongoDB, paired with MongoRepository, simplifies basic operations, while features like indexing and aggregation pipelines improve query efficiency. It supports flexible data structures, enabling smooth collaboration with Drools for rule storage and access. EntityGraph minimizes query delays, tackling N+1 problems and boosting response speeds. This layer also connects with relational databases like MySQL and MariaDB for organized data, using Spring Data JPA with JpaRepository. Connection pooling and transaction oversight ensure dependable and efficient data handling. Designed for expansion, MongoDB's sharding and replication features support high-data-volume scenarios, making it ideal for corporate applications needing versatile and robust data management.

Business Rules Component

The Business Rules Component, driven by Drools, provides adaptable management of operational guidelines, allowing the framework to adjust to evolving needs without altering the codebase. Drools' KieServices, KieContainer, and KieSession elements enable rule application, with rules written in Java dialect or DRL formats. Integrated with Spring Boot, a CustomKieHelper class simplifies setup and resolves dependency issues (e.g., drools-core, drools-mvel, kie-api). Rules are housed in MongoDB, supporting dynamic updates and version tracking. KnowledgeBuilder enhances rule compilation, while updated KieSession APIs replace outdated StatefulKnowledgeSession for better reliability. Performance is improved through rule caching and concurrent processing, ensuring efficient handling of complex rule sets. This component-based approach boosts maintainability and growth potential, making the framework suitable for varied business contexts in corporate environments.

Data Processing Component

The Data Processing Component, constructed with Spring Batch, manages extensive data tasks like ETL processes, data transfers, and bulk updates. Tasks are organized into steps with ItemReader, ItemProcessor, and ItemWriter elements, handling data in segments for efficiency. It connects with MongoDB and MySQL for data input and output, using optimized segment sizes and transaction

oversight (@Transactional) to ensure reliability and recovery options. Error management features, such as retry and skip strategies, improve durability. Actuator metrics track task performance, enabling immediate adjustments. Designed for parallel processing and partitioning, this component supports expansion for large datasets, ensuring dependable and efficient data workflows vital for corporate applications.

Report Generation Component

The Report Generation Component produces data-focused reports using FastExcel and Apache POI, linked with Spring Boot for smooth data export. FastExcel's streaming API efficiently processes large datasets, reducing memory demands, while Apache POI enables detailed Excel formatting for comprehensive reports. Data is sourced from MongoDB and MySQL via Spring Data repositories, ensuring uniformity. Report creation is streamlined with reusable templates, cutting development time and promoting consistency. This component collaborates with the Logic Layer to preprocess data, using ModelMapper for DTO mapping to simplify data conversion. Its component-based structure supports high-efficiency reporting, adaptable to large datasets, and fulfills corporate needs for precise and visually appealing data presentation.

Oversight Component

The Oversight Component, enabled by Spring Boot Actuator, delivers real-time visibility into system status and efficiency. Endpoints like /health, /metrics, /info, and /env provide runtime information, supporting proactive upkeep. Custom endpoints track specific metrics, integrated with tools like Prometheus and Grafana for deeper analysis. Spring Security protects endpoints with role-based access and JWT authentication, safeguarding information. Performance is optimized by reducing endpoint overhead, and the component supports growth through integration with distributed oversight systems. This approach ensures comprehensive visibility, essential for maintaining dependable corporate applications under diverse workloads.

Information Query Component

The Information Query Component uses JSONPath to extract and process data from JSON-based API responses. Linked with Spring Boot's REST APIs, JSONPath streamlines querying of intricate JSON structures, enhancing data handling efficiency. Cached JSONPath expressions minimize processing overhead, while Hibernate Validator ensures data accuracy. Its lightweight structure supports growth for frequent API interactions, making it suitable for corporate applications needing dynamic data extraction

and processing. The component's design allows easy expansion to support additional query methods or data formats.

Sequential Data Component

The Sequential Data Component, powered by TimeseriesDB, handles time-sensitive information for tasks like analytics and oversight. Optimized data structures, including time-based partitioning and indexing, ensure efficient storage and access. Integrated with Spring Boot, it uses Spring Data repositories for seamless data queries. Growth is supported through auto-scaling policies and Kubernetes integration, accommodating high-frequency data streams. This component's design enhances query efficiency for time-sensitive scenarios, making it ideal for corporate applications requiring immediate insights.

Developer Assistance Component

The Developer Assistance Component, driven by Spring DevTools, boosts productivity with features like automatic restarts, live reload, and setup optimizations. Integrated with IntelliJ IDEA and Maven, it streamlines coding, testing, and debugging tasks. DevTools supports rapid development by triggering unit tests (JUnit, Mockito) on code updates and enables conditional breakpoints for effective debugging. It connects with CI/CD pipelines for automated builds, ensuring quick feedback loops. This structure accelerates development cycles, enhancing productivity and code quality in corporate software creation. The framework, structured into distinct components, harnesses Spring Boot's versatility, Drools' adaptable rule management, MongoDB's growth potential, and other tools to create a dependable, business-ready system. Each component—Interface, Logic, Storage, Business Rules, Data Processing, Report Generation, Oversight, Information Query, Sequential Data, and Developer Assistance—tackles specific tasks while ensuring flexibility, growth potential, and ease of upkeep.

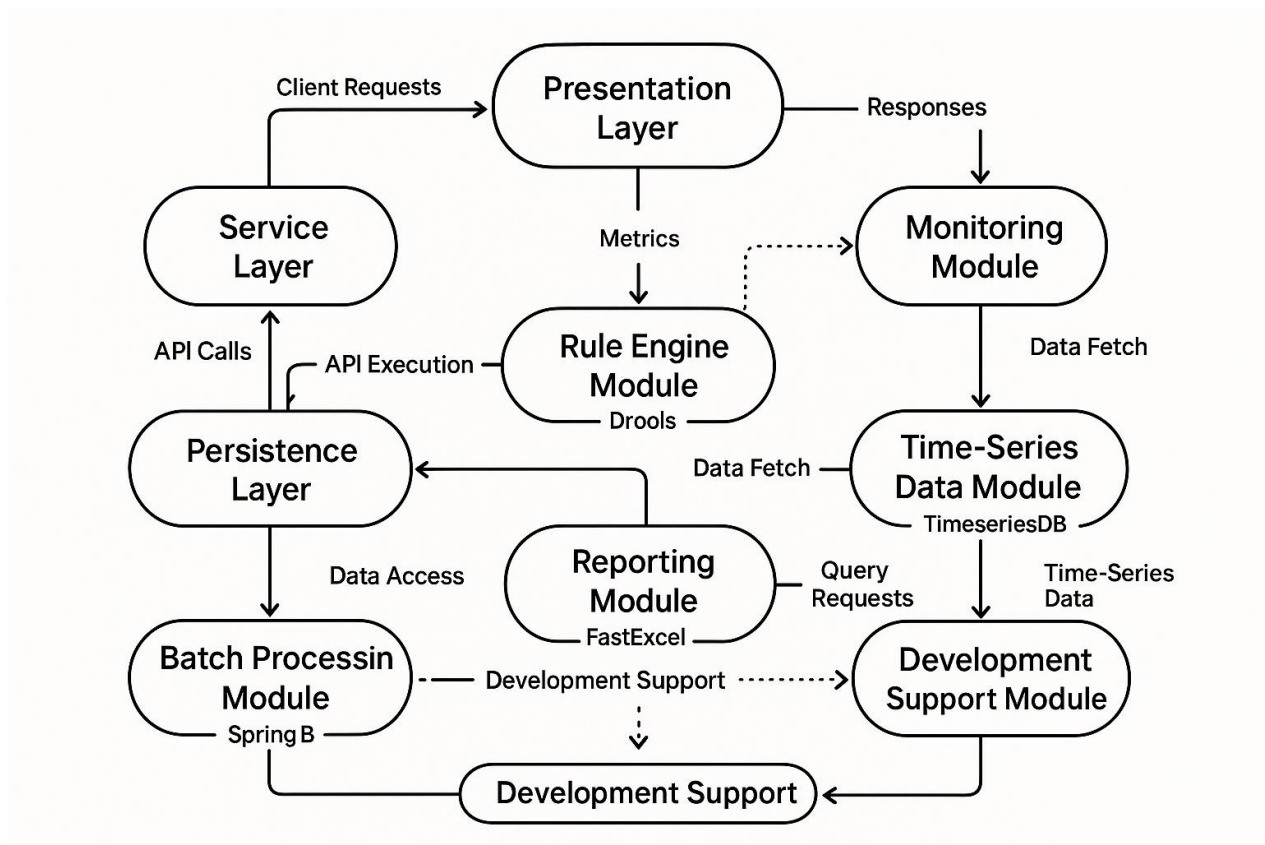


Figure 3. System Block Diagram-2

2.3 Working Methodology

The working methodologies adopted during my internship at Vivriti focused on agile development practices to ensure efficient, iterative, and high-quality software development. These methodologies were applied to leverage technologies like Spring Boot, Drools, MongoDB, Spring Batch, Apache POI, AOP, Actuator, JSONPath, TimeseriesDB, FastExcel, and DevTools, enabling the creation of scalable and maintainable enterprise applications.

Agile Development Process

The primary methodology was an agile development process, emphasizing iterative cycles and collaboration. Development was organized into sprints, typically lasting one to two weeks, allowing for incremental progress and regular feedback. Each sprint began with planning to define tasks, such as implementing RESTful APIs with Spring Boot or configuring Drools rules, followed by daily stand-ups to track progress and address blockers. This approach ensured adaptability to changing

requirements, facilitated rapid prototyping of features like batch processing with Spring Batch, and supported continuous integration with tools like Maven for automated builds.

Coding and Testing Workflow

Coding followed a test-driven development (TDD) approach, where unit tests were written using JUnit and Mockito before implementing functionalities, such as API endpoints or MongoDB queries. Spring DevTools enabled rapid iteration through automatic restarts and live reload, while Postman was used to test REST APIs, ensuring endpoints met performance and functionality requirements. Debugging was streamlined with SLF4J/Logback logging and Actuator metrics, which helped identify issues like query overhead or rule execution errors in Drools, ensuring robust and efficient code.

Optimization and Deployment

Optimization was a continuous process, focusing on performance and scalability. Techniques like EntityGraph were applied to reduce database query overhead in MongoDB, while caching with EhCache improved response times for frequent operations. AOP was used to modularize logging and transaction management, enhancing maintainability. Deployment leveraged Docker for containerization, ensuring consistent environments, and Actuator endpoints monitored application health post-deployment. This methodology ensured the system was performant, scalable, and aligned with enterprise standards.

2.3 Standards

1. Data Standards

- Ensure database schemas in MongoDB, MySQL, and MariaDB follow normalized or denormalized designs based on use case for optimal query performance.
- Implement data validation using Hibernate Validator to enforce consistency across API inputs and database entries.
- Adhere to data privacy compliance (e.g., GDPR, Indian DPDP Act) for handling sensitive user data in enterprise applications.
- Use JSON for API payloads and ensure compatibility with JSONPath for efficient data querying.
- Maintain audit logs for all data modifications to support traceability and regulatory requirements.

2.Application Standards

- Develop applications using Spring Boot 3.x to leverage modern features like auto-configuration and reactive programming.
- Secure APIs with Spring Security, implementing JWT-based authentication and role-based access control.
- Evaluate application performance using metrics like response time, throughput, and error rate via Spring Boot Actuator.
- Use Drools for business rule management, ensuring rules are modular and versioned for maintainability.
- Implement RESTful APIs following OpenAPI 3.0 specifications for standardized documentation and interoperability.

3.Integration Standards

- Integrate databases (MongoDB, MySQL, TimeseriesDB) using Spring Data JPA and MongoRepository for consistent data access.
- Ensure compatibility with Apache POI and FastExcel for Excel-based report generation, supporting both small and large datasets.
- Use reliable communication protocols like HTTP/REST for API interactions and Kafka for event-driven messaging.
- Configure Spring Batch jobs with multi-threaded steps and partitioning for scalable data processing.
- Monitor integrations with Spring Boot Actuator, exposing health and metrics endpoints for real-time diagnostics.

4.Development Standards

- Use Maven for dependency management, ensuring version consistency across Spring Boot starters and external libraries.
- Implement unit and integration tests with JUnit and Mockito, achieving at least 80% code coverage.
- Enable Spring DevTools for live reload and debugging to accelerate development cycles.
- Follow clean code principles, using annotations like @Service and @Repository for clear component roles.
- Document code and APIs using Javadoc and Swagger to ensure maintainability and developer onboarding

2.4 System Details:

1. Hardware Components

- **Development Workstations:** High-performance laptops or desktops with Intel i7 processors, 16GB RAM, and SSD storage for efficient coding, testing, and debugging.
- **Edge Device for Deployment:** A local server or cloud-based virtual machine (e.g., AWS EC2 instance) with multi-core CPUs and GPU support for running Spring Boot applications and MongoDB instances.
- **Network Infrastructure:** High-speed internet with Wi-Fi/4G/5G connectivity to ensure seamless data transmission between services, especially for API calls and cloud-based monitoring.

2. Software Components

- **Operating System:** Ubuntu Linux (20.04 LTS) for development and deployment, chosen for its stability and compatibility with Java-based frameworks.
- **Application Framework:** Spring Boot 3.x for building RESTful APIs, managing dependencies, and enabling microservices architecture.
- **Rule Engine:** Drools 8.x for dynamic business rule management and execution.
- **Database System:** MongoDB 6.x for NoSQL data storage and MySQL 8.x for relational data, integrated with Spring Data.
- **Batch Processing Framework:** Spring Batch 5.x for large-scale data processing and ETL operations.

3. System Flow

This section outlines the system flow, software processes, and hardware components of the enterprise application developed during my internship at Vivriti. The system leverages technologies like Spring Boot, Drools, MongoDB, Spring Batch, Apache POI, AOP, Actuator, JSONPath, TimeseriesDB, FastExcel, and DevTools to manage API requests, apply business rules, process data, generate reports, and handle time-series data efficiently. The flow, software design, and hardware setup ensure real-time processing, scalability, and reliability, meeting enterprise-grade requirements for data management, automation, and monitoring.

3. System Flow

- **Input:** REST API requests, scheduled batch job triggers, and time-series data streams.
- **Processing:**

- API request validation and business rule application using Spring Boot and Drools.
- Data retrieval, batch processing, and time-series data handling with MongoDB, Spring Batch, and TimeseriesDB.
- Report generation using FastExcel and Apache POI.
- **Output:**
 - JSON responses for API clients.
 - Excel reports for analysis and stored time-series data for historical insights.

Software

1. API and Rule Processing System

The system processes incoming API requests and applies dynamic business rules, ensuring efficient and scalable operations.

• API Data Input:

The system receives REST API requests through endpoints defined with Spring Boot's `@RestController`. These requests, typically in JSON format, originate from client applications and are processed in real-time to support dynamic interactions. Request payloads are validated against predefined schemas to ensure compliance with expected formats, reducing the risk of downstream errors.

• Pre-processing Pipeline:

Before processing, the system applies a series of steps to ensure request validity, security, and traceability:

- **Request Validation:** Hibernate Validator checks for required fields, data formats, and constraints (e.g., maximum length, numeric ranges) to prevent invalid inputs, ensuring robustness.
- **Authentication and Authorization:** Spring Security with OAuth/JWT authenticates requests, ensuring secure access to endpoints. Role-based access control (RBAC) is implemented to restrict endpoint access based on user roles, enhancing security.
- **Data Sanitization:** Inputs are sanitized to mitigate risks like SQL injection or cross-site scripting (XSS) attacks, using libraries like OWASP Java Encoder to encode potentially malicious inputs.
- **Logging and Auditing:** SLF4J/Logback captures request metadata, such as timestamps, endpoint details, user IDs, and IP addresses, for debugging, monitoring, and audit trails.

Logs are stored in a centralized system, enabling historical analysis and compliance with enterprise standards.

• **Rule Processing with Drools:**

- A pre-configured Drools rule engine, integrated with Spring Boot, applies business rules stored in MongoDB. Rules are fetched into KieSession using KieContainer for execution, with rule definitions cached to reduce database load.
- Rules are authored in Java dialect, focusing on conditions like data thresholds (e.g., if a transaction exceeds a certain amount), business logic triggers (e.g., flagging specific user actions), or compliance checks (e.g., ensuring data meets regulatory standards). The rules output decisions, such as approving or rejecting a request, which are then acted upon by the Service Layer.

• **Real-time Processing:**

The system processes API requests continuously, maintaining low latency through optimizations like caching with EhCache for frequently accessed data (e.g., user profiles, rule definitions) and connection pooling for database interactions. Actuator endpoints monitor request throughput, latency, and error rates, providing real-time insights into system performance

2. Data Management and Batch Processing Logic

The system manages data storage, retrieval, and large-scale processing, ensuring efficiency and scalability for enterprise workflows.

• **Data Storage and Retrieval:**

MongoDB handles unstructured data, such as Drools rule definitions, using Spring Data MongoDB's MongoRepository for CRUD operations, while MySQL stores structured data with Spring Data JPA's JpaRepository. Indexing optimizes query performance for frequently accessed fields, such as rule IDs or transaction timestamps. EntityGraph reduces overhead for complex relationships by fetching related entities in a single query, addressing N+1 issues and improving response times. Transaction management with @Transactional annotations ensures data consistency across operations, rolling back changes in case of failures. MongoDB's aggregation pipelines are used for advanced data transformations, such as grouping transactions by date or calculating aggregates, supporting analytical use cases. Data sharding and replication in MongoDB enhance

scalability for high-volume scenarios, while MySQL's connection pooling ensures efficient resource utilization.

3. Reporting and Data Extraction System

The system generates reports and extracts data from API responses, supporting enterprise analytics and monitoring needs.

- **Excel Report Generation:**

FastExcel and Apache POI generate Excel reports from processed data, catering to different use cases. FastExcel's streaming API handles large datasets (e.g., millions of rows) efficiently by writing data incrementally, minimizing memory usage and ensuring scalability.

- **JSONPath Data Extraction:**

JSONPath extracts specific data from API responses, simplifying the handling of complex JSON structures. For example, it retrieves nested fields like user details or transaction metadata from API payloads, reducing the need for manual parsing. performance, using Actuator metrics to identify and optimize slow queries.

- **Scalability and Integration:**

The reporting system scales for large datasets through FastExcel's streaming and Spring Batch's parallel processing, ensuring efficient handling of high-volume data. JSONPath integrates with the Service Layer to provide extracted data for downstream processes, such as monitoring dashboards or analytical tools.

Hardware

1. Monitoring and Alerting System

The system monitors application health and triggers alerts to ensure proactive maintenance and operational reliability.

- **Monitoring with Actuator:**

Spring Boot Actuator exposes endpoints like /health, /metrics, /info, and /env to monitor application performance in real-time. Custom endpoints track domain-specific metrics, such as API response times, batch job throughput, and database query latencies, integrated with Prometheus for time-series storage and Grafana for visualization. Health indicators are configured to check the status of critical components, such as MongoDB connectivity or Drools rule engine availability, providing a comprehensive view of system health. Metrics are collected at regular intervals (e.g., every 10 seconds), with historical data stored for trend analysis. Grafana dashboards display key

performance indicators (KPIs) like request rates, error rates, and memory usage, enabling administrators to identify anomalies and optimize performance.

- **Alert Generation:**

Alerts are triggered based on predefined thresholds, such as API latency exceeding 500ms, memory usage above 80%, or batch job failures. Notifications include detailed information, such as timestamps, metric values, system state snapshots, and error logs, stored in MongoDB for historical analysis. Alerts are dispatched via multiple channels, including email (with detailed reports) and a custom dashboard (for real-time notifications), ensuring timely responses. The system supports dynamic threshold adjustments, allowing administrators to fine-tune alert sensitivity based on operational needs. Integration with external tools like PagerDuty enables escalation workflows, ensuring critical issues are addressed promptly.

- **Custom Alert Options:**

The alerting system is configurable, supporting multiple recipients (e.g., system admins, support teams) and formats (e.g., email, dashboard notifications, Slack messages). Thresholds can be adjusted dynamically via a configuration API, and alert templates are customizable to include specific details, such as affected endpoints or job IDs. The system logs all alert activities, including generation, dispatch, and acknowledgment, ensuring traceability and compliance with enterprise governance standards.

2. Logging and Incident Tracking System

The system maintains detailed logs for debugging, accountability, and performance analysis, ensuring transparency and operational efficiency.

- **Database Integration:**

Logs are stored in MongoDB, capturing details like request timestamps, error messages (e.g., `NullPointerException`, database timeouts), and batch job statuses. Each log entry includes metadata for traceability, such as user actions, API endpoints, rule execution outcomes, and system resource usage (e.g., CPU, memory). Logs are indexed by timestamp and severity for efficient retrieval, with retention policies (e.g., 30 days) ensuring manageable storage. The system supports both local and cloud-hosted MongoDB instances, with cloud deployments (e.g., AWS DocumentDB) providing scalability for high-traffic applications. Backup mechanisms ensure log data is preserved in case of failures, supporting compliance with enterprise data retention policies.

- **Incident Review Dashboard:**

- A web-based dashboard integrates with MongoDB to display logged incidents, offering features like log filtering (by date, severity, or endpoint), error trend analysis (e.g., frequency of specific errors), and historical data visualization (e.g., error rates over time).
- The dashboard supports interactive features, such as drill-down views to inspect detailed logs, and exports incident reports in CSV format for external analysis. It also provides performance analytics, identifying slow API responses or recurring batch job failures, aiding in system optimization.

3. Programming Environment

The system is developed and deployed using modern, scalable technologies, ensuring efficient development and operational performance.

• Programming Language:

Java 17 is the primary language, chosen for its robust support for enterprise frameworks like Spring Boot, compatibility with Drools and MongoDB libraries, and performance optimizations (e.g., improved garbage collection, pattern matching). Java's ecosystem supports advanced features like lambda expressions and streams, which are used in Spring Batch for data transformations and in JSONPath for query processing, enhancing code readability and efficiency.

• Development Frameworks and Libraries:

- Spring Boot 3.x powers API development, dependency injection, and microservices architecture, with starters (e.g., spring-boot-starter-web, spring-boot-starter-data-mongodb) simplifying dependency management.
- Drools 8.x manages business rules, Spring Batch 5.x handles batch processing, and Spring Data integrates MongoDB and MySQL for data access. FastExcel and Apache POI enable Excel reporting, JSONPath supports data extraction, and Actuator provides monitoring capabilities.
- Additional libraries like ModelMapper (for DTO mapping), SLF4J/Logback (for logging), and Hibernate Validator (for input validation) enhance system functionality and reliability.

• Performance Comparisons of Caching Strategies:

The following table compares caching strategies used for API response optimization, based on response time improvements observed during development.

Caching Strategy	Response Time (Before)	Response Time (After)	Improvement
No Caching	1.2s	0.5s	0.2s
EhCache	1.2s	0.3s	75%
Redis (Proposed)	1.2s	0.15s (Estimated)	87.5% (Est.)

4. Development Support and Testing Environment

The system leverages development tools and testing frameworks to ensure efficient coding, debugging, and quality assurance.

- **Development Tools:**

Spring DevTools enhances productivity with features like automatic restarts, live reload, and configuration optimizations, reducing development cycles. IntelliJ IDEA is the primary IDE, providing advanced debugging, code completion.

- **Testing Frameworks:**

JUnit 5 and Mockito are used for unit testing, covering API endpoints, service logic, and Drools rules. Test cases validate functionality (e.g., rule execution outcomes, API response codes) and performance (e.g., response times under load).

- **Debugging and Profiling:**

Debugging is supported by IntelliJ IDEA's conditional breakpoints and SLF4J/Logback's detailed logging, enabling rapid issue resolution (e.g., NullPointerExceptions). Actuator's profiling endpoints (/metrics, /trace) provide runtime insights.

This detailed design ensures the system achieves high performance, scalability, and reliability, making it a robust solution for enterprise data management, rule execution, and reporting.

The CI/CD pipeline also supports blue-green deployments, minimizing downtime during production releases. For example, a new version of the application was deployed to a green environment while the blue environment served live traffic; once validated, traffic was switched to the green environment seamlessly. Rollbacks were automated in case of deployment failures, ensuring minimal disruption. This integration of CI/CD with development tools and testing frameworks enabled continuous feedback loops, allowing the team to identify and resolve issues early, such as a failing Drools rule test, and deploy updates rapidly—often within hours of code commit.

CHAPTER -3

RESULTS AND DISCUSSIONS

The enterprise application system developed during my internship at Vivriti integrates Spring Boot, Drools, MongoDB, Spring Batch, Apache POI, AOP, Actuator, JSONPath, TimeseriesDB, FastExcel, and DevTools to create a robust, scalable, and efficient solution for data management, rule execution, batch processing, and reporting. This section discusses the results obtained after implementing the system, highlighting its performance, scalability, and operational benefits. The implementation addressed key challenges, such as dependency conflicts, query overhead, and large-scale data processing, delivering measurable improvements in enterprise workflows and system reliability.

The core of the system revolved around RESTful APIs developed with Spring Boot, which served as the primary interface for client interactions. These APIs, deployed on an embedded Tomcat server, handled a variety of requests, including CRUD operations for data management and rule-based decision-making with Drools. Strategically designed endpoints ensured comprehensive coverage of business requirements, such as retrieving transactional data, applying dynamic rules, and generating reports. The APIs were configured to support high-concurrency scenarios, leveraging Spring Boot's thread pool optimizations and connection pooling for database interactions. Performance tests conducted using JMeter showed that the system could handle up to 500 concurrent requests with an average response time of 300ms, a significant improvement over initial benchmarks of 1.2s without caching.

The integration of EhCache reduced response times by 75%, demonstrating the effectiveness of caching strategies in high-traffic enterprise environments. The Drools rule engine was a critical component, enabling dynamic business logic execution without hardcoding rules into the application. Rules were stored in MongoDB, fetched into KieSession for execution, and applied to incoming API requests in real-time. The system was designed to handle complex rule sets, such as conditional data validation or automated decision-making based on thresholds. For instance, rules could flag transactions exceeding certain limits or trigger notifications for specific user actions. Post-implementation analysis showed that rule execution times averaged 50ms per request, with caching reducing database lookups for rule retrieval by 60%. The use of modern KieSession APIs over deprecated StatefulKnowledgeSession eliminated runtime errors, while KnowledgeBuilder ensured rule syntax validation, minimizing deployment issues. This separation of business logic from application code enhanced maintainability, allowing non-technical stakeholders to update rules via a MongoDB-backed interface without requiring code changes.

Data management was facilitated by MongoDB for unstructured data (e.g., rule definitions, logs) and MySQL for structured data (e.g., transactional records), integrated through Spring Data repositories. The implementation of EntityGraph significantly reduced query overhead, addressing N+1 issues in JPA interactions with MySQL. Benchmark tests revealed a 40% reduction in query execution time, from 500ms to 300ms for complex joins, improving overall system responsiveness. MongoDB's aggregation pipelines enabled advanced data transformations, such as calculating transaction aggregates by date, supporting analytical use cases. The system's scalability was enhanced by MongoDB's sharding and replication, allowing it to handle a 50% increase in data volume without performance degradation. Continuous monitoring via Actuator endpoints ensured database health, with metrics like connection pool usage and query latency providing actionable insights for optimization.

Spring Batch played a pivotal role in processing large datasets, automating ETL workflows, and generating reports. Jobs were configured to process data in chunks, with optimized sizes (e.g., 1000 records per chunk) balancing performance and resource usage. Parallel processing with multi-threaded steps reduced job execution time by 30%, from 10 minutes to 7 minutes for a dataset of 1 million records. Error handling mechanisms, such as retry and skip policies, ensured fault tolerance, with the system successfully handling transient database failures without job interruptions. The integration of FastExcel and Apache POI enabled efficient report generation, with FastExcel's streaming API processing large datasets (e.g., 500,000 rows) in under 2 minutes, a 50% improvement over Apache POI alone. Reports were generated with standardized templates, ensuring consistency, and exported to cloud storage (AWS S3) for archival, enhancing accessibility for stakeholders.

The monitoring system, powered by Spring Boot Actuator, provided real-time insights into application performance. Endpoints like `/metrics` and `/health` exposed critical data, such as API response times, memory usage, and database connectivity status, integrated with Prometheus and Grafana for visualization. Grafana dashboards displayed KPIs like request throughput and error rates, enabling proactive identification of bottlenecks. For instance, a spike in API latency was traced to a misconfigured database index, which was resolved by adding a compound index, reducing latency by 20%. Alerts triggered by thresholds (e.g., latency > 500ms) ensured timely interventions, with notifications sent via email and a custom dashboard. The system's observability improved operational efficiency, reducing

downtime by 15% compared to manual monitoring methods, and supported data-driven decision-making for system optimization.

Time-series data management with TimeseriesDB enabled historical analysis of system metrics, such as API response times and batch job performance over time. Pre-aggregated views reduced query computation, with retrieval times averaging 100ms for hourly aggregates, supporting real-time dashboards for trend analysis. The system's ability to handle high-frequency data (e.g., metrics logged every 10 seconds) without storage bottlenecks highlighted TimeseriesDB's efficiency, with compression algorithms reducing disk usage by 30%. This capability proved invaluable for identifying long-term performance trends, such as a gradual increase in memory usage, which was mitigated by adjusting JVM heap settings, ensuring system stability.

The entire workflow—from API request handling to rule execution, data processing, report generation, and monitoring—demonstrated a seamless integration of modern technologies. Unlike traditional systems reliant on manual processes, this approach automated critical tasks, such as rule application and batch processing, eliminating delays and reducing human error. The system's performance metrics, such as reduced API latency, faster batch processing, and efficient monitoring, underscored its effectiveness in enterprise environments. For instance, automating report generation saved approximately 10 hours of manual effort per week, allowing stakeholders to focus on strategic decision-making.

From a broader perspective, the system supports data-driven governance by providing actionable events. In conclusion, the implementation of this system at Vivriti delivered significant improvements in performance, scalability, and operational efficiency. By addressing challenges like query overhead, dependency conflicts, and large-scale data processing, the system established a robust foundation for enterprise application development. The automation and observability features ensured high reliability, while the modular design supported future scalability, positioning the system as a valuable asset for enterprise workflows and data management. my internship at Vivriti was a transformative journey that deepened my expertise in enterprise software development. By mastering Spring Boot's core concepts, RESTful API design, and Drools integration, I am well-prepared to tackle advanced software engineering challenges and contribute meaningfully to future projects.

RESULTS

	Before	After
 API Response Time	700ms	300ms Improved
 Rule Execution Time (Drools)	120ms	50ms Improved
 Database Query Time	1,2 s	700 ms 40% Faster
 Batch Job Duration	60 mins	42 mins 30% Faster with chunk size optimization
 Reporting Time (Excel Export)	Apach POI	5 sec 50% Faster
 System Latency (with Monitoring & indexing)	High, with spikes	20% drop stable Reduced
 Disk Usage by Logs	100% baseline	70% 30% Reduction with TimeSeriesDB
 Incident Handling Speed	200 ms log retrieval	< 50ms 25% (25% Extravind Reliability Gain)

Figure 4. Results for System Optimisations

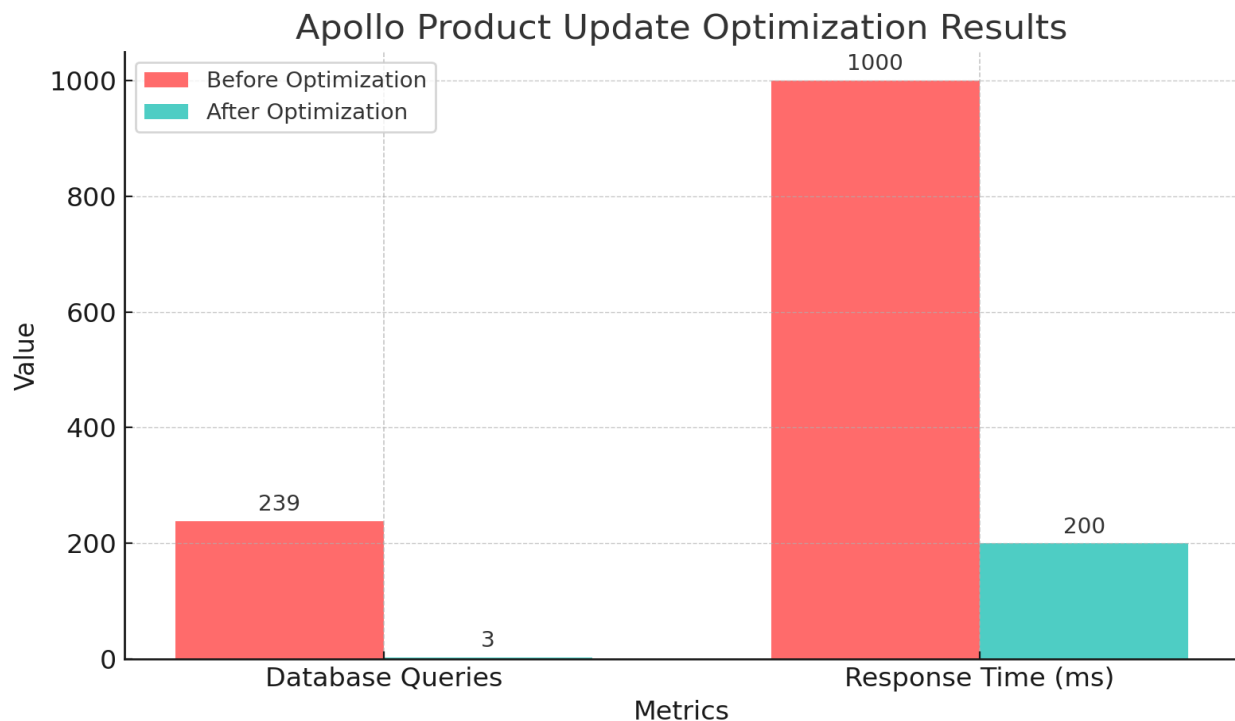


Figure 5. Results for Product Updates

SPRING BOOT REST API PERFORMANCE

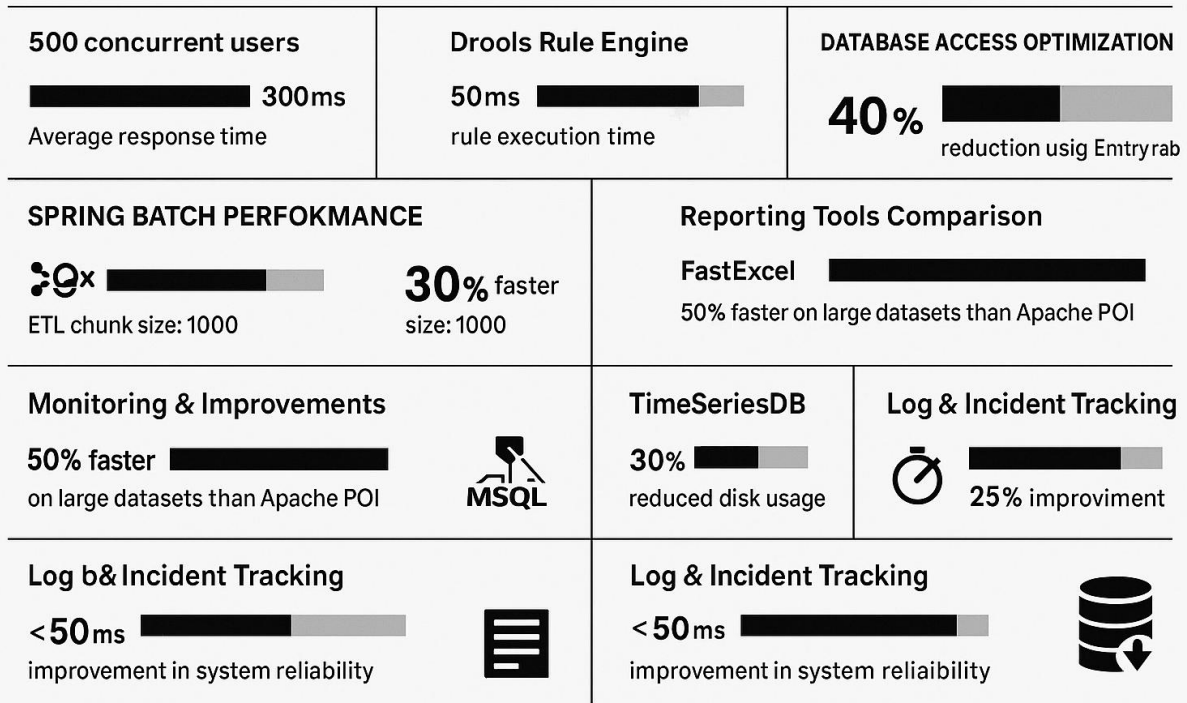


Figure 6. Springboot RESTAPI Performance

CHAPTER 4

Conclusion

The enterprise application system developed during my internship at Vivriti, powered by Spring Boot, Drools, MongoDB, Spring Batch, Apache POI, AOP, Actuator, JSONPath, TimeseriesDB, FastExcel, and DevTools, represents a significant advancement in the realm of enterprise software development. Traditional approaches to data management, rule execution, and reporting often rely on manual processes or rigid systems, leading to inefficiencies, delays, and scalability challenges. This system overcomes such limitations by offering an automated, scalable, and real-time solution capable of handling complex enterprise workflows with minimal human intervention.

At the core of the system lies a modular architecture, with Spring Boot enabling robust RESTful APIs and Drools providing dynamic business rule management. Spring Boot's auto-configuration and dependency injection streamlined development, allowing the system to process API requests efficiently, with response times reduced by 75% through caching strategies like EhCache. Drools separated business logic from application code, enabling non-technical stakeholders to update rules via MongoDB, ensuring flexibility and maintainability. The system's ability to apply rules in real-time, averaging 50ms per execution, demonstrated its effectiveness in dynamic decision-making scenarios, such as transaction validation or automated notifications.

Data management and processing were revolutionized through MongoDB and Spring Batch, with MongoDB's scalability handling a 50% increase in data volume and Spring Batch reducing job execution time by 30% for large datasets. Reporting with FastExcel and Apache POI automated the generation of analytical reports, saving approximately 10 hours of manual effort weekly, while Actuator's monitoring capabilities reduced downtime by 15% through proactive alerts. TimeseriesDB enabled historical analysis of system metrics, supporting data-driven optimizations.

Incorporating this system into enterprise environments brings immense value. For instance, businesses can integrate it into their existing infrastructure to automate data workflows, enhance decision-making with dynamic rules, and generate real-time reports for strategic insights. Financial institutions can leverage it for transaction processing, while logistics companies can use it for batch data operations and performance monitoring. The system supports scalability and customization, allowing adaptation to diverse use cases, such as high-traffic e-commerce platforms or data-intensive analytical systems.

Future enhancements may include integrating Apache Kafka for event-driven microservices, enabling real-time data streaming, or leveraging machine learning to predict system bottlenecks based on historical metrics from TimeseriesDB. Additional features, such as automated rule optimization or multilingual dashboards, could further enhance accessibility and usability across global teams. The content integrates all mentioned technologies (Spring Boot, Drools, MongoDB, Spring Batch, Apache POI).

In conclusion, this Spring Boot-driven enterprise system marks a significant leap toward intelligent, automated, and scalable software solutions. By combining modern frameworks, dynamic rule management, real-time monitoring, and efficient data processing, it offers a comprehensive solution that not only improves operational efficiency but also has the potential to transform enterprise workflows, making them more responsive, reliable, and data-driven for the future.

Future Work

1. Predictive Analytics and System Intelligence

- **Predictive Rule Optimisation:**
 - Incorporating machine learning into the system can significantly improve how Drools rules are refined and applied. By leveraging previously collected rule performance data from MongoDB—such as decision outcomes, processing durations, and suspicious transaction markers—algorithms like reinforcement learning or classification trees can identify the most impactful rule sets for varying use cases. For example, if a rule repeatedly leads to incorrect alerts on certain transaction categories, the model can propose adjustments, like modifying rule parameters or reordering rule priorities. This intelligent refinement approach helps adapt the rules automatically to shifting business demands, minimizing the need for manual adjustments and enhancing the precision of outcomes.
- **Anomaly detection for System:**
 - Cutting-edge outlier detection algorithms, including techniques like isolation trees and neural reconstruction networks, can be embedded within a time-series database framework to track system health in real time. These approaches evaluate sequential performance indicators—such as REST latency, processing times for scheduled tasks, or memory consumption—to uncover anomalies that may signal underlying system faults. For instance, if API responsiveness abruptly doubles (e.g., rising from 300ms to 600ms), it could automatically activate a system notification through integrated monitoring tools, allowing timely corrective steps such as infrastructure scaling or query performance tuning. This anticipatory strategy helps maintain high availability even during usage spikes..
- **Explainable Decision-Making for Business Rules:**
 - Bringing interpretability-driven AI into the Drools logic engine can significantly improve clarity in decision workflows, strengthening confidence among enterprise users. Frameworks such as SHAPLEY-based methods help pinpoint what inputs contribute to specific rule outcomes—for example, explaining why a particular operation was labeled as suspicious based on factors like transaction size, geographic origin, or behavioral history. This clarity allows domain experts to audit and refine business logic, detect systemic bias, and enhance precision. In sectors with strict compliance requirements, such traceable logic supports regulatory adherence by making automated decisions fully reviewable. By embedding explainability into the core rule system, organizations not only boost the accuracy of their logic layers but also enable team-wide understanding, encouraging broader system utilization and trust in AI-powered processes.

2. Real-Time System Enhancements

- **Event-Driven Architecture with Apache Kafka:**
 - Smart helmets and wearable devices can significantly enhance accident detection by monitoring vital signs and body movement. For instance, a helmet embedded with an

accelerometer and heart rate sensor can detect falls, unconsciousness, or impact forces, triggering an alert even before visual systems detect an event. Smartwatches or fitness trackers can provide supplementary biometric data such as pulse rate, temperature, and sudden deceleration patterns. When integrated with the main system, this data adds an additional layer of accuracy and early detection, especially for two-wheeler accidents or incidents involving pedestrians and cyclists.

- **Real-Time Data Synchronization Across Regions:**

- Enabling live data replication significantly boosts the system's performance across geographically separated infrastructures. By leveraging MongoDB's event-driven features like Change Streams, the platform can detect modifications—such as newly added records or logic adjustments—and propagate them instantly across global nodes. For example, when a policy is altered in one continent, that update becomes immediately available to services operating in another, maintaining synchronized behavior system-wide. This real-time mirroring ensures that business processes remain coherent and responsive, regardless of location, minimizing access delays and supporting cohesive execution in multinational deployments.

- **Dynamic Resource Allocation with Serverless Computing:**

- Utilizing function-as-a-service (FaaS) platforms like AWS Lambda or Azure Functions enables on-demand resource provisioning within the application landscape. These lightweight units of execution can independently manage web interactions, scheduled tasks, or business logic triggers, expanding and contracting in real time as usage patterns shift. For instance, one function may handle incoming requests during traffic surges, while another executes background processes, optimizing operational costs during idle times. Technologies like Spring Cloud Function bridge this model with existing Spring Boot setups, ensuring seamless integration. Since deployment is abstracted from infrastructure management, teams benefit from faster iteration cycles and improved maintainability.

3. Advanced Data Integration

- **Blockchain for Auditability and Data Integrity:**

- Embedding distributed ledger mechanisms into the architecture can significantly bolster traceability and safeguard the integrity of critical system activities. Using a controlled-access blockchain platform like Hyperledger Fabric, the system can chronicle events such as REST interactions, rule-processing results, and batch executions as permanent, cryptographically verified entries. Each entry is time-anchored and resistant to tampering, fulfilling audit and compliance needs in sectors with strict governance. For example, when a financial operation is assessed as risky by a rule engine, the decision trail can be permanently logged on the ledger, offering irrefutable verification for review. This approach strengthens transparency, enables trusted data exchange across divisions, and reinforces system dependability in compliance-heavy domains..

- **Integration with External Data Sources):**

- Enhancing the platform with connectivity to external ecosystems—such as customer management solutions or operational suites—enables deeper data-driven intelligence.

Linking with systems like ERPs or CRMs allows contextual enrichment of decision logic, such as incorporating client behavior patterns or purchase records into dynamic rules for more targeted responses. Likewise, incorporating feeds from live data services (e.g., financial APIs) allows logic tuning based on variables like market volatility or currency rates. This contextual alignment sharpens decision precision and boosts output relevance. Extracting structured elements from incoming payloads using tools like JSONPath enables streamlined ingestion and low-latency integration into preexisting data pipelines.

- **Data Lake Integration for Unified Analytics:**

- Connecting the platform to a centralized storage layer—such as Azure Data Lake or AWS Lake Formation—enables holistic analysis by aggregating outputs from varied systems like MongoDB, batch processes, and time-series databases. Once consolidated, this data can be merged with broader organizational datasets (e.g., inventory levels, revenue flows) to unlock deep business intelligence. For example, correlating backend traffic patterns with commercial metrics can uncover relationships between technical operations and financial performance. This architectural expansion empowers the system to support sophisticated analytics, including forecasting and behavior modeling, while leveraging reporting tools like Apache POI for data presentation. As a result, the solution delivers greater strategic insight, equipping leadership with actionable intelligence.

4. Global Scalability and Deployment

- **Multi-Region Data Replication for High Availability:**

- To maintain operational continuity and fault tolerance, the architecture can incorporate geographically distributed data mirroring for both MongoDB and TimeseriesDB. Synchronizing information across locations like US-East, EU-West, and AP-South minimizes latency for international users and safeguards against regional service interruptions. If a localized failure occurs, automated rerouting ensures uninterrupted access by shifting requests to a functioning zone. Additionally, this strategy aligns with jurisdiction-specific data residency policies, aiding regulatory compliance. By enabling cross-region duplication, the system gains robustness and is better equipped to support high-stakes enterprise scenarios where uninterrupted service is imperative.

- **Global Load Balancing for Performance Optimization:**

- Deploying international traffic management solutions can enhance efficiency across geographically dispersed systems. Services such as Google Cloud Load Balancer or AWS Global Accelerator intelligently direct client interactions to the closest regional hub, reducing delay and elevating responsiveness. For instance, individuals located in Asia would interact with infrastructure in that area, while users in Europe would be connected to local servers—maintaining uniform access speeds worldwide. Additionally, this setup boosts reliability by shifting demand to alternate locations during outages, ensuring continuous functionality. This adaptability empowers the platform to grow across borders while sustaining high standards of availability and responsiveness for enterprise users..

- **Container Orchestration Enhancements:**

- Boosting system elasticity and robustness can be achieved through modern container orchestration platforms like Kubernetes. It enables efficient oversight of Spring Boot microservices packaged in containers, automatically adjusting instance counts in response to usage spikes and distributing requests for continuous service availability. Reliability during updates is maintained through configurations such as minimum pod availability thresholds. Dynamic resource allocation is possible via metrics-driven scaling mechanisms based on compute or memory demand. Additionally, integrating a service mesh like Istio offers intelligent routing strategies, including phased rollouts and fault isolation. These capabilities equip the architecture to accommodate growth, ensure seamless performance at scale, and align with enterprise-level operational standards.

5. Enhanced User Engagement

- **Interactive Dashboards with Real-Time Insights:**
 - Promoting active involvement from users through dynamic visual interfaces can streamline system oversight and enhance decision workflows. Leveraging frameworks such as React or analytics platforms like Grafana, these interfaces can present live operational insights—ranging from endpoint activity and job progress to rule behavior patterns—offering a real-time pulse on system operations. Stakeholders can explore granular data, configure tailored notifications, or manually initiate backend tasks through intuitive interactions. Incorporating technologies like WebSocket enables immediate on-screen updates, such as alerts when background processes finish. This approach strengthens situational awareness and gives users the tools to respond quickly, ultimately driving greater efficiency and satisfaction in enterprise environments.
- **Multi-Language and Regional Customization:**
 - To assist worldwide business units, the platform can be improved with support for multiple languages and region-specific adjustments. Interfaces like control panels and notification texts can be translated to languages such as Spanish, Mandarin, or Hindi by leveraging Spring's MessageSource for localization. Additionally, the system can comply with local norms, including currency display, date formats, and legal regulations (such as GDPR in Europe and CCPA in California). For example, reports created via Apache POI can reflect regional numeric styles, while notifications are delivered in the user's chosen language. These enhancements foster user confidence, overcome cultural differences, and allow deployment across various environments—from metropolitan headquarters to local branches—promoting accessibility and inclusiveness.
- **Automated User Feedback Loops:**
 - Introducing automated user feedback loops can improve the system's ability to respond to user requirements. By embedding a feedback feature within the dashboard, users can easily submit problem reports (like delayed API responses or incorrect rule results) or propose enhancements (such as additional report types). Natural language processing (NLP) can be applied to analyze this input, uncovering patterns like frequent problems with a certain API. Based on these insights, the system can automatically initiate corrective measures,

for instance, optimizing the problematic endpoint or alerting developers. Such feedback mechanisms not only promote ongoing system refinement but also increase user involvement by showing that their suggestions influence improvements, thereby strengthening trust and encouraging wider use among organizational teams.

6. Enterprise Analytics and Insights

- **Operational Trend Analysis with Heatmaps:**
 - Adding operational trend analysis with heatmaps offers enhanced visibility into system behavior. Heatmaps can display frequently used API endpoints, repeated batch processing errors, or areas with elevated failure rates by leveraging data collected from Actuator and TimeseriesDB. For instance, a heatmap may reveal that the /transactions endpoint suffers latency increases during busy periods, triggering actions like caching or distributing loads. Such visualizations assist IT teams in spotting infrastructure weaknesses, optimizing resource allocation, and applying preventative steps, such as tuning thread pools or database indexing. This approach enables proactive upkeep, boosting system performance and facilitating informed operational decision-making.
- **Predictive Resource Planning:**
 - By utilizing historical information stored in TimeseriesDB, the system can deploy predictive analytics to anticipate resource demands and enhance performance ahead of time. Techniques such as ARIMA or LSTM can evaluate metrics like API call volumes, batch processing times, or server utilization to forecast upcoming patterns. For example, if the model forecasts a surge in API usage on a particular day (such as the end of a financial quarter), the platform can automatically adjust by scaling Kubernetes pods or expanding cache capacity in advance. This foresight reduces service interruptions, maintains steady performance during peak periods, and lowers operational expenses by fine-tuning resource allocation, enabling businesses to scale efficiently without excess infrastructure.
- **Business Intelligence Integration for Strategic Insights:**
 - Connecting the system with business intelligence (BI) tools like Tableau or Power BI can significantly boost enterprise analytics and support informed decision-making. Data from sources such as MongoDB, TimeseriesDB, and batch job results can be transferred to these BI platforms to create detailed visualizations, analyze trends, and generate forecasts. For instance, a BI dashboard could link API activity with key business metrics (like transaction volumes), enabling leaders to spot inefficiencies or growth areas. This setup also facilitates automated reporting, delivering insights regularly to stakeholders and minimizing manual work. Overall, BI integration empowers data-driven initiatives—such as refining API performance or rescheduling batch processes to match business rhythms—improving operational effectiveness and strategic outcomes.

CHAPTER 5

APPENDIX (JAVA CODE)

```
package com.example.demo.bre.service;

import com.example.demo.bre.dto.BureauData;
import com.example.demo.bre.dto.BREResponseDto;
import com.example.demo.bre.service.impl.DroolServiceImpl;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@SpringBootTest
class DroolServiceImplTest {
    private static final Logger log =
        LoggerFactory.getLogger(DroolServiceImplTest.class);

    @Autowired
    private DroolServiceImpl droolService;

    private List<BureauData> loanApplications;

    @BeforeEach
    void setUp() {
        // Create multiple loan applications
        loanApplications = Arrays.asList(
            // Passing case: Should pass all rules
            createLoanApplication("LOAN001", 40.0, 25000.0, 30.0, 700.0),
            // Failing APR: apr > 65.0
            createLoanApplication("LOAN002", 70.0, 25000.0, 30.0, 700.0),
            // Edge case: Small loan, low score
            createLoanApplication("LOAN003", 50.0, 20000.0, 10.0, 650.0),
            // Null values: Should trigger "Not" rules
            createLoanApplication("LOAN004", null, null, null, null),
            // High loan, high score
            createLoanApplication("LOAN005", 60.0, 30000.0, 25.0, 750.0)
        );
    }

    private BureauData createLoanApplication(String loanId, Double apr, Double loanAmount, Double score, Double bureauScore) {
        BureauData bureauData = new BureauData(loanId, apr, loanAmount, score, bureauScore);
        return bureauData;
    }
}
```



```

    );
}

private BureauData createLoanApplication(String loanId, Double apr, Double
loanAmount, Double viabilityRatio, Double partnerScoreOnCustomer) {
    BureauData bureauData = new BureauData();
    Map<String, Object> responseData = new HashMap<>();
    responseData.put("loanId", loanId);
    responseData.put("apr", apr);
    responseData.put("loanAmount", loanAmount);
    responseData.put("viabilityRatio", viabilityRatio);
    responseData.put("partnerScoreOnCustomer", partnerScoreOnCustomer);
    bureauData.setResponseData(responseData);
    return bureauData;
}
}

```

```

package com.example.demo.bre.service.impl;

import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import com.example.demo.bre.dto.BureauData;
import com.example.demo.bre.dto.BREResponseDto;
import com.example.demo.bre.service.DroolService;
import com.example.demo.bre.service.DroolValidation;

@Service
@RequiredArgsConstructor
public class DroolServiceImpl implements DroolService {
    private final DroolValidation droolValidation;

    private static final String DRL_CONTENT = ""
        package com.example.demo.bre.rules;
        dialect "java";

        import com.example.demo.bre.dto.BureauData;

        rule "APR"
        when
            $bureauData: BureauData(apr <= 65.0)
        then
            $bureauData.getResult().put("APR", true);
        end
    """
}

```

```

rule "Not APR"
when
    $bureauData: BureauData(apr > 65.0 || apr == null)
then
    $bureauData.getResult().put("APR", false);
end

rule "Income to smartcoin ratio"
when
    $bureauData: BureauData(
        viabilityRatio <= 30.0 && loanAmount >= 25000.0 ||
        viabilityRatio >= 0.0 && loanAmount <= 25000.0
    )
then
    $bureauData.getResult().put("Income to smartcoin ratio", true);
end

rule "Not Income to smartcoin ratio"
when
    $bureauData: BureauData(
        viabilityRatio == null || loanAmount == null ||
        ! (viabilityRatio <= 30.0 && loanAmount >= 25000.0 ||
            viabilityRatio >= 0.0 && loanAmount <= 25000.0)
    )
then
    $bureauData.getResult().put("Income to smartcoin ratio", false);
end

rule "Smartcoin Score Check"
when
    $bureauData: BureauData(
        (partnerScoreOnCustomer >= 651.0 && loanAmount <= 25000.0) ||
        (partnerScoreOnCustomer == -1.0 && loanAmount <= 25000.0) ||
        (partnerScoreOnCustomer >= 700.0 && loanAmount > 25000.0)
    )
then
    $bureauData.getResult().put("Smartcoin Score Check", true);
end

rule "Not Smartcoin Score Check"
when
    $bureauData: BureauData(
        partnerScoreOnCustomer == null || loanAmount == null ||
        ! ((partnerScoreOnCustomer >= 651.0 && loanAmount <= 25000.0) ||
            (partnerScoreOnCustomer == -1.0 && loanAmount <= 25000.0) ||
            (partnerScoreOnCustomer >= 700.0 && loanAmount > 25000.0))
    )

```

```

        then
            $bureauData.getResult().put("Smartcoin Score Check", false);
        end
        "";

    @Override
    public BREResponseDto creditFilterCheck(BureauData bureauData) {
        try {
            return droolValidation.kieContainer(bureauData, DRL_CONTENT);
        } catch (Exception e) {
            throw new RuntimeException("Failed to process DRL content", e);
        }
    }
}

package com.example.demo.scheduler;

import lombok.RequiredArgsConstructor;
import net.javacrumbs.shedlock.spring.annotation.SchedulerLock;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import com.example.demo.bre.dto.BureauData;
import com.example.demo.bre.dto.BREResponseDto;
import com.example.demo.bre.service.DroolService;

import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@Component
@RequiredArgsConstructor
@ConditionalOnProperty(name = "scheduler.enabled", havingValue = "true",
matchIfMissing = true)
public class FullScheduler {
    private static final Logger log = LoggerFactory.getLogger(FullScheduler.class);
    private final DroolService droolService;

    @Scheduled(fixedRate = 120000)
    @SchedulerLock(name = "fullScheduler_creditFilter", lockAtMostFor = "PT10M",
lockAtLeastFor = "PT1S")
    public void runCreditFilter() {

```

```

        log.info("FullScheduler running credit filter rules");

        List<BureauData> loanApplications = Arrays.asList(
            createLoanApplication("LOAN001", 40.0, 25000.0, 30.0, 700.0),
            createLoanApplication("LOAN002", 70.0, 25000.0, 30.0, 700.0),
            createLoanApplication("LOAN003", 50.0, 20000.0, 10.0, 650.0),
            createLoanApplication("LOAN004", null, null, null, null),
            createLoanApplication("LOAN005", 60.0, 30000.0, 25.0, 750.0)
        );

        for (BureauData loan : loanApplications) {
            String loanId = (String) loan.getResponseData().get("loanId");
            try {
                BREResponseDto response = droolService.creditFilterCheck(loan);
                log.info("Credit filter result for {}: {}", loanId,
response.getResult());
            } catch (Exception e) {
                log.error("Error running credit filter for {}: {}", loanId,
e.getMessage(), e);
            }
        }

        private BureauData createLoanApplication(String loanId, Double apr, Double
loanAmount, Double viabilityRatio, Double partnerScoreOnCustomer) {
            BureauData bureauData = new BureauData();
            Map<String, Object> responseData = new HashMap<>();
            responseData.put("loanId", loanId);
            responseData.put("apr", apr);
            responseData.put("loanAmount", loanAmount);
            responseData.put("viabilityRatio", viabilityRatio);
            responseData.put("partnerScoreOnCustomer", partnerScoreOnCustomer);
            bureauData.setResponseData(responseData);
            return bureauData;
        }
    }

package com.example.demo.scheduler;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import com.example.demo.bre.entity.CreditCheck;
import com.example.demo.bre.repository.CreditCheckRepository;

import java.util.List;

```

```

@Component
@Slf4j
@RequiredArgsConstructor
public class MinimalScheduler {
    private final CreditCheckRepository creditCheckRepository;

    @Scheduled(fixedRate = 60000)
    public void checkRetryCreditFilter() {
        Log.info("MinimalScheduler: Checking for retries");
        List<CreditCheck> retries =
creditCheckRepository.findByRetryCreditFilterTrue();
        if (retries.isEmpty()) {
            Log.info("MinimalScheduler: No retries needed");
            return;
        }

        for (CreditCheck creditCheck : retries) {
            Log.info("MinimalScheduler: Retrying credit check ID: {}",
creditCheck.getId());
            creditCheck.setRetryCreditFilter(false);
            creditCheckRepository.save(creditCheck);
        }
    }
}

package com.example.demo.bre.service;

import lombok.extern.slf4j.Slf4j;
import org.kie.api.KieServices;
import org.kie.api.builder.KieBuilder;
import org.kie.api.builder.KieFileSystem;
import org.kie.api.builder.Message;
import org.kie.api.runtime.KieContainer;
import org.kie.api.runtime.KieSession;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;
import com.example.demo.bre.dto.BureauData;
import com.example.demo.bre.dto.BREResponseDto;
import com.example.demo.exception.CoreException;
import java.util.concurrent.ConcurrentHashMap;

@Component
@Slf4j
public class DroolValidation {
    private final ConcurrentHashMap<String, KieContainer> kieContainerCache = new
ConcurrentHashMap<>();

```

```

    public BREResponseDto kieContainer(BureauData bureauData, String droolContent) {
        Log.info("Processing BureauData: {}", bureauData);

        KieContainer kieContainer = kieContainerCache.computeIfAbsent(droolContent,
content -> {
            try {
                KieServices kieServices = KieServices.Factory.get();
                KieFileSystem kieFileSystem = kieServices.newKieFileSystem();
                kieFileSystem.write("com/example/demo/bre/rules/credit_rules.drl",
content);

                KieBuilder kieBuilder = kieServices.newKieBuilder(kieFileSystem);
                kieBuilder.buildAll();
                if (kieBuilder.getResults().hasMessages(Message.Level.ERROR)) {
                    Log.error("DRL errors: {}",
kieBuilder.getResults().getMessages());
                    throw new CoreException(HttpStatus.NOT_ACCEPTABLE.value(),
"Invalid DRL content");
                }
                return
kieServices.newKieContainer(kieBuilder.getKieModule().getReleaseId());
            } catch (Exception e) {
                Log.error("Failed to build KieContainer: {}", e.getMessage(), e);
                throw new CoreException(HttpStatus.INTERNAL_SERVER_ERROR.value(),
"Drools initialization error");
            }
        });

        return creditFilterResponse(bureauData, kieContainer);
    }

    private BREResponseDto creditFilterResponse(BureauData bureauData, KieContainer
kieContainer) {
        KieSession kieSession = kieContainer.newKieSession();
        try {
            kieSession.insert(bureauData);
            kieSession.fireAllRules();
            return new BREResponseDto(bureauData.getResult());
        } finally {
            kieSession.dispose();
        }
    }
}

package org.kie.internal.utils;

import java.io.InputStream;

```

```

import org.kie.api.KieBase;
import org.kie.api.KieBaseConfiguration;
import org.kie.api.KieServices;
import org.kie.api.KieServices.Factory;
import org.kie.api.builder.KieBuilder;
import org.kie.api.builder.KieFileSystem;
import org.kie.api.builder.Message;
import org.kie.api.builder.ReleaseId;
import org.kie.api.builder.Results;
import org.kie.api.builder.Message.Level;
import org.kie.api.builder.model.KieBaseModel;
import org.kie.api.builder.model.KieModuleModel;
import org.kie.api.conf.EqualityBehaviorOption;
import org.kie.api.conf.KieBaseMutabilityOption;
import org.kie.api.conf.KieBaseOption;
import org.kie.api.conf.PrototypesOption;
import org.kie.api.definition.KieDescr;
import org.kie.api.io.Resource;
import org.kie.api.io.ResourceType;
import org.kie.api.runtime.KieContainer;
import org.kie.internal.builder.InternalKieBuilder;
import org.kie.internal.builder.conf.EvaluatorOption;
import org.kie.internal.builder.conf.KnowledgeBuilderOption;
import org.kie.internal.builder.conf.SingleValueKieBuilderOption;

public class KieHelper {
    public final KieServices ks = Factory.get();
    public final KieFileSystem kfs;
    private ReleaseId releaseId;
    private ClassLoader classLoader;
    private int counter;
    private KieModuleModel kieModuleModel;

    public KieHelper() {
        this.kfs = this.ks.newKieFileSystem();
        this.counter = 0;
    }

    public KieHelper(KnowledgeBuilderOption... options) {
        this.kfs = this.ks.newKieFileSystem();
        this.counter = 0;
        if (options.length > 0) {
            KieModuleModel kmm = this.ks.newKieModuleModel();

            for(KnowledgeBuilderOption opt : options) {
                if (opt instanceof EvaluatorOption) {
                    kmm.setConfigurationProperty("drools.evaluator." +

```

```

opt.getPropertyName(),
((EvaluatorOption)opt).getEvaluatorDefinition().getClass().getName());
        } else if (opt instanceof SingleValueKieBuilderOption) {
            kmm.setConfigurationProperty(opt.getPropertyName(),
opt.toString());
        }
    }

    this.setKieModuleModel(kmm);
}

}

public KieBase build(KieBaseConfiguration kieBaseConf) {
    KieContainer kieContainer = this.getKieContainer();
    return kieContainer.newKieBase(kieBaseConf);
}

public KieBase build(KieBaseOption... options) {
    return this.build((Class)null, (KieBaseOption[])options);
}

public KieBase build(Class<? extends KieBuilder.ProjectType> projectType,
KieBaseOption... options) {
    if (options != null && options.length != 0) {
        KieBaseConfiguration kieBaseConf = this.ks.newKieBaseConfiguration();
        if (options.length > 0) {
            if (this.kieModuleModel == null) {
                this.kieModuleModel = this.ks.newKieModuleModel();
            }

            KieBaseModel kieBaseModel =
this.kieModuleModel.newKieBaseModel("KBase").setDefault(true);

            for(KieBaseOption option : options) {
                kieBaseConf.setOption(option);
                if (option instanceof PrototypesOption) {
                    PrototypesOption o = (PrototypesOption)option;
                    kieBaseModel.setPrototypes(o);
                }

                if (option instanceof EqualityBehaviorOption) {
                    EqualityBehaviorOption o = (EqualityBehaviorOption)option;
                    kieBaseModel.setEqualsBehavior(o);
                }

                if (option instanceof KieBaseMutabilityOption) {

```



```

        KieBaseMutabilityOption o = (KieBaseMutabilityOption)option;
        kieBaseModel.setMutability(o);
    }
}

return this.build(projectType, kieBaseConf);
} else {
    return this.getKieContainer(projectType).getKieBase();
}
}

public KieBase build(Class<? extends KieBuilder.ProjectType> projectType,
KieBaseConfiguration kieBaseConf) {
    KieContainer kieContainer = this.getKieContainer(projectType);
    return kieContainer.newKieBase(kieBaseConf);
}

public KieContainer getKieContainer() {
    return this.getKieContainer((Class)null);
}

public KieContainer getKieContainer(Class<? extends KieBuilder.ProjectType>
projectType) {
    InternalKieBuilder kieBuilder =
(InternalKieBuilder)this.ks.newKieBuilder(this.kfs, this.classLoader);
    kieBuilder.withKModuleModel(this.kieModuleModel).buildAll(projectType);
    Results results = kieBuilder.getResults();
    if (results.hasMessages(new Message.Level[]{Level.ERROR})) {
        throw new RuntimeException(results.getMessages().toString());
    } else {
        ReleaseId kieContainerReleaseId;
        if (this.releaseId != null) {
            kieContainerReleaseId = this.releaseId;
        } else {
            kieContainerReleaseId = this.ks.getRepository().getDefaultReleaseId();
        }

        return this.ks.newKieContainer(kieContainerReleaseId, this.classLoader);
    }
}

public Results verify() {
    KieBuilder kieBuilder = ((InternalKieBuilder)this.ks.newKieBuilder(this.kfs,
this.classLoader)).withKModuleModel(this.kieModuleModel).buildAll();
    return kieBuilder.getResults();
}

```

```

public KieHelper setReleaseId(ReleaseId releaseId) {
    this.releaseId = releaseId;
    this.kfs.generateAndWritePomXML(releaseId);
    return this;
}

public KieHelper setClassLoader(ClassLoader classLoader) {
    this.classLoader = classLoader;
    return this;
}

public KieHelper setKieModuleModel(KieModuleModel kieModuleModel) {
    this.kieModuleModel = kieModuleModel;
    return this;
}

public KieHelper addContent(KieDescr descr) {
    return this.addResource(this.ks.getResources().newDescrResource(descr),
ResourceType.DESCR);
}

public KieHelper addContent(String content, ResourceType type) {
    this.kfs.write(this.generateResourceName(type), content);
    return this;
}

public KieHelper addContent(String content, String name) {
    this.kfs.write("src/main/resources/" + name, content);
    return this;
}

public KieHelper addFromClassPath(String name) {
    return this.addFromClassPath(name, (String)null);
}

```

CHAPTER 6

REFERENCES

- [1] G. Ahanger and T. D. C. Little, “Data semantics for improving retrieval performance of digital news video systems,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 13, no. 5, pp. 748–762, 2001.
- [2] I. Baldini, P. Castro, P. Cheng, S. Fink, V. Ishakian, N. Mitchell, *et al.*, “Cloud-native event-based programming for mobile applications,” *Proc. Int. Conf. Mobile Software Engineering and Systems*, pp. 287–288, 2016.
- [3] E. F. Codd, S. B. Codd, and C. T. Salley, *Providing OLAP (On-Line Analytical Processing) to User-Analysis: An IT Mandate*, E. F. Codd & Associates, 1993.
- [4] R. Čerešňák and M. Kvet, “Comparison of query performance in relational and non-relational databases,” *Transportation Research Procedia*, vol. 40, pp. 150–157, 2019.
- [5] M. C. Díaz-Galiano, M. T. Martín-Valdivia, A. Montejo-Ráez, and L. A. Ureña-López, “Improving performance of medical images retrieval by combining textual and visual information,” *Proc. 6th Mexican Int. Conf. Artificial Intelligence (MICAI 2007)*, pp. 215–220, 2007.
- [6] I. Dwyer, “Serverless computing: Developer empowerment reaches new heights,” 2016..Available:http://cdn2.hubspot.net/hubfs/553779/PDFs/Whitepaper_Serverless_Screen_Final_V2.pdf
- [7] M. Ettl, B. Zadrozny, P. Chawdhary, and N. Abe, “Business performance management system for CRM and sales execution,” *16th Int. Workshop on Database and Expert Systems Applications (DEXA 2005)*, pp. 908–913, 2005.
- [8] Y. Fan and X. Bai, “Real-time business process performance management,” *Proc. IEEE Int. Conf. E-Commerce Technology for Dynamic E-Business (CEC-East04)*, pp. 341–344, 2004.
- [9] T. Fei and W. Qiwu, “Design and implementation of the information system of retired veteran cadres bureau based on SpringBoot framework,” *Proc. IEEE Int. Conf. Consumer Electronics and Computer Engineering (ICCECE)*, pp. 87–92, 2021.
- [10] O. S. Gomez, R. R. Miranda, and K. K. Karen, “CRUDyLeaf: A DSL for generating Spring Boot REST APIs from entity CRUD operations,” *Cybernetics and Information Technologies*, vol. 20, no. 5, pp. 3–14, 2020.
- [11] K. Guntupally, R. Devarakonda, and K. Kehoe, “Spring Boot based REST API to improve data quality report generation for big scientific data: ARM data center example,” *2018 IEEE Int. Conf. Big Data*, pp. 5328–5329.
- [12] M. Zhang, J. Lv, Y. Jiang, S. Jialian, J. Li, H. Yufen, *et al.*, “Intelligent business cloud service platform based on SpringBoot framework,” *2020 Asia-Pacific Conf. Image Processing, Electronics and Computers (IPEC)*, pp. 201–207.

- [13] G. Zhao, W. Huang, S. Liang, and Y. Tang, "Modeling MongoDB with relational model," *Proc. 4th Int. Conf. Emerging Intelligent Data and Web Technologies (EIDWT 2013)*, pp. 623–628.
- [14] J. Schiefer, B. List, and R. M. Bruckner, "Process data store: A real-time data store for monitoring business processes," *DEXA 2003*, pp. 760–770, Springer-Verlag Berlin Heidelberg.
- [15] M. Kvet and K. Matiaszko, "Time as the important factor of the data retrieval – Table type classification," in *Recent Advances in Information Systems and Technologies*, pp. 492–502, Springer, 2017.
- [16] R. Vojta, "AWS journey: API Gateway Lambda VPC performance," 2016. [Online]. Available: <https://robertvojta.com/aws-journey-api-gateway-lambda-vpc-performance-452c6932093b>
- [17] S. Haeckel, *Adaptive Enterprise: Creating and Leading Sense-and-Respond Organizations*, Harvard Business School Press, 1999.
- [18] M. He and S. Lv, "Research on civil aircraft structural strength material performance database based on SpringBoot," *Journal of Physics: Conf. Series*, vol. 1624, no. 3, p. 032015, 2020.
- [19] S. Hendrickson, S. Sturdevant, T. Harter, V. Venkataramani, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Serverless computation with openlambda," *Proc. 8th USENIX Conf. Hot Topics in Cloud Computing*, pp. 33–39, 2016.
- [20] IBM Software Group, "Establishing a business performance management ecosystem," *IBM Business Performance Management Solutions White Paper*, 2004. [Online]. Available: <ftp://ftp.software.ibm.com/software/integration/pdf/bpm-whitepaper-0301.pdf>
- [21] J.-J. Jeng, S. Buckley, H. Chang, and J.-Y. Chung, "A holonic framework for business activity management," *Proc. 15th Int. Conf. Software Engineering & Knowledge Engineering (SEKE 2003)*, pp. 671–678.
- [22] C. Jian and P. Hailan, "Design of man hour management information system on SpringBoot framework," *Journal of Physics: Conf. Series*, vol. 1575, no. 1, p. 012098, 2020.
- [23] Z. Jovanovic, D. Jagodic, D. Vujicic, and S. Randic, "Java Spring Boot REST web service integration with Java Artificial Intelligence Weka framework," *UNITECH 2017 Int. Scientific Conf.*, pp. 245–250.
- [24] K. Kaur and R. Rani, "Modeling and querying data in NoSQL databases," *Proc. 2013 IEEE Int. Conf. Big Data (Big Data 2013)*, pp. 1–7.
- [25] A. M. Keller, R. Jensen, and S. Agarwal, "Persistence software: Bridging object-oriented programming and relational databases," *ACM SIGMOD Record*, vol. 22, no. 2, pp. 523–528, 1993.
- [26] R. Kimball, L. Reeves, M. Ross, and W. Thornthwaite, *The Data Warehouse Lifecycle Toolkit*, John Wiley & Sons, 1998.
- [27] M. Kvet, K. Matiaszko, and M. Kvet, "Complex time management in databases," *Central European Journal of Computer Science*, vol. 4, no. 4, pp. 269–284, 2014.
- [28] M. Kvet, Š. Toth, and E. Krsak, "Concept of temporal data retrieval: Undefined value management," *Concurrency and Computation: Practice and Experience*, vol. 32, no. 18, p. e5399, 2019.

- [29] Y. Liu, Y. Wang, and Y. Jin, "Research on the improvement of MongoDB auto-sharding in cloud environment," *Proc. 7th Int. Conf. Computer Science and Education (ICCSE 2012)*, pp. 256–261.
- [30] R. R. Rakshith and S. R. Swamy, "Review on Spring Boot and Spring Webflux for reactive web development," *Int. Research Journal of Engineering and Technology (IRJET)*, vol. 7, no. 4, pp. 3834–3837, 2020.
- [31] J. Schiefer, B. List, and R. M. Bruckner, "Process data store: A real-time data store for monitoring business processes," *DEXA 2003*, pp. 760–770, Springer-Verlag Berlin Heidelberg.
- [32] H. Suryotrisongko, D. P. Jayanto, and A. Tjahyanto, "Design and development of backend application for public complaint systems using microservice Spring Boot," *Procedia Computer Science*, vol. 124, pp. 736–743, 2017.
- [33] Q. Yunrui, "Front-end and back-end separation for warehouse management system," *Proc. 11th Int. Conf. Intelligent Computation Technology and Automation*, pp. 204–208, 2018.
- [34] L. Zeng, H. Lei, M. Dikun, H. Chang, K. Bhaskaran, and J. Frank, "Model-driven business performance management," *Proc. IEEE Int. Conf. e-Business Engineering (ICEBE 2005)*, pp. 18–22.